

Public Key Kryptographie

Vorlesungsskript

WS 96/97

Willi Geiselman

Vorabversion

Ausgearbeitet von

Pawel Wocjan

Vorabversion¹

¹Korrektur- und Ergänzungsvorschläge werden mit Interesse entgegengenommen

Inhaltsverzeichnis

Einführung, Terminologie	4
Terminologie	4
Aufgaben der Kryptographie	5
Algorithmen und Schlüssel	5
Algorithmen ohne Schlüssel	5
Algorithmen mit Schlüssel	5
Kryptoanalyse	6
Sicherheit von Algorithmen	7
Aufwand von Angriffen	7
1 Protokolle	8
1.1 Einführung	8
1.2 Protokoll mit Notar	9
1.3 Typen von Angriffen gegen Protokolle	10
1.4 Eine erste Public-Key-Idee (Merkles Puzzle)	10
1.5 Einwegfunktionen	11
1.6 Hashfunktionen	11
1.7 Hybrid-Kryptosystem	12
1.8 Digitale Signatur	12
1.8.1 Signatur mit Hilfe eines Notars und eines symmetrischen Krypto- systems	13
1.8.2 Signatur mit Public-Key-Verfahren	13
1.8.3 Signatur mit Hashfunktionen	13
1.8.4 Angriffe gegen Hashfunktionen	14
1.9 Protokolle für Schlüsselaustausch	15

1.9.1	Protokoll mit einem symmetrischen Verfahren und einer Schlüsselzentrale (SZ)	15
1.9.2	Protokoll mit Public-Key und einer Schlüsselzentrale	15
1.9.3	Man-in-the-middle Angriff	16
1.9.4	Interlock-Protokoll	16
1.9.5	Schlüsselaustausch mit Signaturen	17
1.10	Zero Knowledge Protokolle	18
2	Public Key Algorithmen	23
2.1	Knapsack	23
2.2	RSA-Verfahren	26
2.3	Diffie-Hellman-Verfahren	27
2.4	ElGamal-Verfahren	28
2.5	McEliece-Kryptosystem	30
3	Mathematische Grundlagen	32
3.1	Primzahlen	32
3.2	Quadratische Reste	35
3.3	Primtests	38
3.3.1	Pseudoprimzahlen	39
3.3.2	Probabilistischer Primtest nach Solovay, Strassen	41
3.3.3	Probabilistischer Primtest nach Rabin und Miller	44
3.4	Faktorisierungsalgorithmen mit Gruppen	45
3.4.1	Pollard $p - 1$ -Methode	45
3.4.2	Faktorisierung mit elliptischen Kurven	47
3.5	Faktorisierung mit quadratischen Kongruenzen	51
3.5.1	Grundlagen der Laufzeitanalyse	51
3.5.2	Dixons Algorithmus	53
3.5.3	Laufzeitanalyse	54
3.5.4	Quadratischer Siebalgorithmus	55

4	Das diskrete Logarithmusproblem	59
4.1	Einführung	59
4.2	Baby-Step Giant-Step Algorithmus	59
4.3	Index Calculus	60
4.3.1	Endliche Körper	61
4.3.2	Index-Calculus für $\mathbb{F}_p^*$	64
4.3.3	Index-Calculus für $\mathbb{F}_{p^n}$	64
4.4	Coppersmith-Algorithmus für $\mathbb{F}_{2^n}$	66
5	Authentifikation	69
5.1	Authentifikation mit Einwegfunktionen	69
5.2	Authentifikation mit Hilfe von Public Key	70
5.3	Authentifikation mit Schlüsselaustausch	71
5.4	Digital Cash	72
5.4.1	Erste Protokolle	73
5.4.2	Hilfsprotokolle	75
5.4.3	Ein vollständiges Protokoll	76

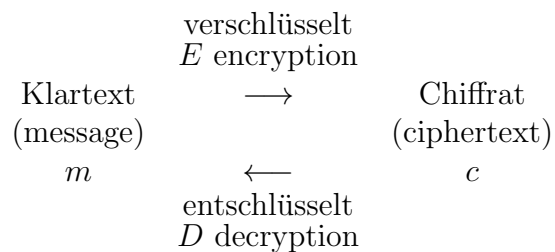
Einführung, Terminologie

Terminologie

Das kleinst mögliche Modell in der Kryptographie hat folgende Form:

- Sender Alice, Empfänger Bob;
- Nachricht oder Klartext m , Chiffre c ;
- Verschlüsselung E (encryption), Entschlüsselung D (decryption).

Alice will Bob eine Nachricht m schicken. Alice will erreichen, daß ihre Nachricht sicher übertragen wird, d.h. ein Unbefugter darf die Nachricht nicht lesen oder verändern können. Um dies zu erreichen, muß die Nachricht von Alice verschlüsselt werden, so daß sie nur von Bob entschlüsselt werden kann.



Mathematisch kann der Sachverhalt wie folgt beschrieben werden. Die Funktion E ist eine injektive Funktion von der Menge der Klartexte in die Menge der Chiffre, D ist eine surjektive Funktion in der entgegengesetzten Richtung.

$$\begin{aligned} E(m) &= c \\ D(c) &= m \\ D(E(m)) &= m \\ E(D(c)) &= c \end{aligned}$$

Aufgaben der Kryptographie

Kryptographie oder auch *Kryptologie* (aus dem Griechischen *krypós* „versteckt“ und *lógos* „Wort“) ist die Wissenschaft, die sich mit der sicheren (geheimen) Kommunikation befaßt. Die Vertraulichkeit ist nur eine der Aufgaben, die die Kryptographie heute erfüllen muß. Üblicherweise werden die folgenden vier Eigenschaften betrachtet:

- **Vertraulichkeit:**
Geheimhaltung der Nachricht.
- **Integrität:**
Empfänger kann verifizieren, daß die Nachricht nicht verändert wurde.
- **Verbindlichkeit:**
Unabstreitbarkeit, Sender kann die Herkunft nicht ableugnen.
- **Authentizität:**
eindeutige Herkunft, Nachricht ist wirklich vom Sender.

Algorithmen und Schlüssel

Ein *kryptographischer Algorithmus* ist die Realisierung einer mathematischen Funktion, die zur Ver- und Entschlüsselung benutzt wird. (Im allgemeinen gibt es zwei verschiedene Funktionen zur Ver- und Entschlüsselung.)

Algorithmen ohne Schlüssel

- Sicherheit beruht auf der Geheimhaltung des Algorithmus.
- Der Algorithmus kann schlecht auf Güte überprüft werden, falls kein kompetentes Mitglied in der Gruppe vorhanden ist, das den Algorithmus benutzt.
- Personen in mehreren Gruppen benötigen mehrere Algorithmen.
- Nach einem erfolgreichen Angriff oder nachdem ein Mitglied die Gruppe verlassen hat, muß der Algorithmus vollständig geändert werden.

Algorithmen mit Schlüssel

Es wird ein Algorithmus verwendet, der von Kryptographen geprüft oder standardisiert werden kann. Die Sicherheit wird durch die Wahl eines geheimen Schlüssels erreicht. Es gibt zwei Klassen von Algorithmen mit Schlüssel.

1. Symmetrische Algorithmen

Die Schlüssel zur Ver- und Entschlüsselung sind einfach auseinander berechenbar, meistens sind sie identisch. Sender und Empfänger einigen sich vor dem Senden von Nachrichten auf den (die) Schlüssel.

- **Stromchiffren:** Nachrichten werden bit- bzw. byteweise verarbeitet. Das Chiffirat eines Zeichens hängt von den vorigen Zeichen bzw. von der Stelle im Text ab.
- **Blockchiffren:** Nachrichten werden blockweise (64 bit, 128 bit) verarbeitet. Ein Klartextblock wird unabhängig von der Stelle seines Auftretens auf ein Chiffirat abgebildet.

2. Asymmetrische Algorithmen (oder Public-Key-Algorithmen)

Es werden unterschiedliche Schlüssel verwendet. Der Schlüssel zur Entschlüsselung läßt sich nicht mit vertretbarem Aufwand aus dem Schlüssel zum Verschlüsseln berechnen. Der Schlüssel zum Verschlüsseln heißt *öffentlicher Schlüssel*, der andere *geheimer Schlüssel*.

Kryptoanalyse

Das Ziel der *Kryptoanalyse* ist es, aus einem Chiffirat c die zugehörige Nachricht m ohne Kenntnis des geheimen Schlüssels zu gewinnen oder den geheimen Schlüssel zu finden. Die Grundannahme ist dabei, daß der Algorithmus bekannt ist. Es gibt vier Typen von Angriffen:

1. Ciphertext-Only

- Geg: Einige Chiffirate, die mit demselben Schlüssel und Algorithmus verschlüsselt wurden.
Ziel: Klartexte und/oder Schlüssel zu finden.

2. Known-plaintext

- Geg: Paare von Klartext und Chiffirat.
Ziel: Finden des Schlüssels (um später unbekannte Klartexte zu finden).

3. Chosen-plaintext

- Geg: Klartexte können frei gewählt werden, deren Chiffirate dann berechnet werden.
Ziel: Schlüssel zu finden.

4. Chosen-ciphertext

- Geg: Chiffirate können frei gewählt werden, die dann entschlüsselt werden (bei Public-Keyverfahren sinnvoll).
Ziel: Schlüssel (geheimen Schlüssel bei PK) zu finden.

Alle Kryptosysteme können durch einen Known-plaintext-Angriff gebrochen werden, indem man alle möglichen Schlüssel ausprobiert. Diese Art, den Schlüssel aus einem Klartext-Chiffre-Paar zu gewinnen, heißt *Brute-force-Angriff*. Oft ist dieser Angriff wegen der Redundanz in der Nachricht (Sprache, bekannter Anfang, Anrede, usw.) auch möglich, wenn nur Chiffre gegeben sind. Die Kryptographie sucht Algorithmen, die mit den zur Verfügung stehenden Mitteln (heutige oder zukünftige) nicht gebrochen werden können.

Sicherheit von Algorithmen

Es gibt vier Kategorien, die das Versagen eines kryptographischen Algorithmus beschreiben.

1. **Vollständiger Zusammenbruch**
Der (geheime) Schlüssel wurde gefunden.
2. **Globale Entschlüsselung** (global deduction)
Es wurde ein anderer Algorithmus zum Entschlüsseln gefunden, ohne den Schlüssel zu kennen.
3. **Lokale Entschlüsselung**
Es wurden von einigen Chiffren die Klartexte gefunden.
4. **Informationsgewinn**
Teile oder Information über den Klartext und/oder den Schlüssel wurden gefunden.

Aufwand von Angriffen

Der Aufwand der Angriffe kann auf drei verschiedene Weisen gemessen werden:

1. **Rechenkapazität:**
Zeit bzw. Anzahl der Rechenschritte.
2. **Speicherkomplexität:**
benötigter Speicher.
3. **Datenkomplexität:**
benötigte Anzahl an Chiffren bzw. Chiffre-Klartext-Paaren.

Kapitel 1

Protokolle

1.1 Einführung

Ein Protokoll besteht aus einer Reihe von Schritten, die nacheinander ausgeführt werden. Es sind mindestens zwei Teilnehmer am Protokoll beteiligt. Durch das Protokoll soll ein gewisser Zweck erfüllt werden. Weiterhin müssen noch folgende Bedingungen erfüllt sein:

- Das Protokoll ist jedem Teilnehmer bekannt.
- Jeder Teilnehmer ist mit dem Protokoll einverstanden.
- Der Ablauf des Protokolls muß eindeutig sein.
- Das Protokoll muß vollständig sein (es ist in allen Fällen genau bestimmt, was zu tun ist).

Das Ziel eines kryptographischen Protokolls ist es, Abhören und Betrügen zu verhindern. Außerdem darf kein Teilnehmer mehr erfahren können als das, was im Protokoll vorgesehen ist.

Beispiel 1.1 *Informale Protokolle aus dem täglichen Leben:*

- *Telefonische Bestellung von Gütern;*
- *Bezahlen mit Kreditkarte;*
- *Bürgerentscheid;*
- *Kartenspiele.*

Diese Protokolle funktionieren meistens, da man die Teilnehmer kennt, sieht oder hört. Computer brauchen im Gegensatz zu Menschen formale Protokolle.

1.2 Protokoll mit Notar

Ein *Notar* ist eine vertrauenswürdige Instanz, die keinen der Teilnehmer bevorzugt. Mit Hilfe einer solchen Instanz lassen sich recht einfach Protokolle angeben.

Beispiel 1.2 (Verkauf eines Autos mit Hilfe eines Notars) *Der Verkauf kann wie folgt abgewickelt werden:*

1. *Alice will Bob ein Auto verkaufen.*
2. *Bob will mit einem Scheck bezahlen.*
3. *Alice gibt den KFZ-Schein dem Notar.*
4. *Bob gibt den Scheck an Alice, die ihn dann bei der Bank einlöst.*
5. *Falls der Scheck gedeckt war, gibt der Notar den KFZ-Schein an Bob.*
6. *Falls aber der Scheck nicht gedeckt war, muß Alice dies dem Notar beweisen und erhält daraufhin den KFZ-Schein vom Notar zurück.*

Eine zweite Lösung wäre ein von der Bank zertifizierter Scheck. In diesem Fall könnte man ohne Notar auskommen.

Das obige Beispiel läßt sich auf Rechnernetzwerke übertragen. Ein wesentlicher Nachteil ist es, daß bei jeder Transaktion der Notar involviert ist. Dieser Engpaß kann durch den Einsatz mehrerer Notare behoben werden, was aber die Kosten steigert. Eine andere Möglichkeit, diesen Engpaß zu beheben, ist es, das Protokoll wie folgt in zwei Teile zu zerlegen:

- **Normalfall**
 - Alice und Bob handeln einen Vertrag aus,
 - Alice unterschreibt,
 - Bob unterschreibt.
- **Ausnahmefall** (Vertrag wird nicht eingehalten)
 - Alice und Bob erscheinen vor Gericht,
 - Alice und Bob erläutern den Fall,
 - Richter fällt eine Entscheidung.

Dieses Protokoll verhindert nicht einen Betrug, hat aber einen *Abschreckungseffekt*. Das Beste, was man manchmal auch erreichen kann, ist ein *abgesichertes Protokoll*. Dieses Protokoll garantiert, daß keiner der Teilnehmer betrügen kann. Sobald sich ein Partner nicht an das Protokoll hält (betrügen will), bemerkt das der andere Teilnehmer und bricht ab.

1.3 Typen von Angriffen gegen Protokolle

- **passiver Angriff:**
Ein Unbeteiligter hört die Nachrichten ab und versucht daraus Information zu gewinnen.
- **aktiver Angriff:**
Ein Unbeteiligter ändert die Nachrichten (fängt beispielsweise Nachrichten ab und schickt andere).
- **passiver Betrug:**
Ein Teilnehmer des Protokolls versucht zusätzliche Informationen zu gewinnen (beispielsweise wählt er bei einer vom Protokoll verlangten Wahl von Zufallszahlen geschickt und nicht zufällig).
- **aktiver Betrug:**
Ein Teilnehmer hält sich nicht an das Protokoll (er gibt beispielsweise einen falschen Namen an).

1.4 Eine erste Public-Key-Idee (Merkles Puzzle)

Merkles Puzzel basiert auf der Idee, Rätsel zu erzeugen, die für den Sender Alice und den Empfänger Bob in vertretbarer Zeit zu lösen sind, nicht aber für einen Abhörer Eve. Damit soll das Ziel erreicht werden, daß vor der Kommunikation kein gemeinsamer Schlüssel zwischen Alice und Bob ausgetauscht werden muß.

Vorgehensweise:

1. Bob erzeugt $2^{20} (\approx 10^6)$ Nachrichten der Form „**Rätselnr.:** x_i , **Schlüssel:** y_i “, wobei x_i und y_i jeweils paarweise verschiedene Zufallszahlen sind. Die y_i haben eine Länge von mindestens 40 bit. Bob verschlüsselt diese Nachrichten mit einem symmetrischen Verfahren, wobei er verschiedene Schlüssel der Länge 20 bit benutzt, und schickt alles an Alice.
2. Alice wählt eine der verschlüsselten Nachrichten zufällig aus und entschlüsselt sie durch einen Brute-force-Angriff (Aufwand ungefähr 10^6 Schritte). Alice erhält nach der Entschlüsselung die beiden Zufallszahlen x_0 und y_0 .
3. Alice verschlüsselt die Nachricht m mit dem Schlüssel y_0 und schickt das Chifftrat zusammen mit x_0 im Klartext an Bob.
4. Bob wählt das zu x_0 gehörende y_0 aus seiner Datenbank und entschlüsselt das Chifftrat von Alice mit dem Schlüssel y_0 .

Aufwandsbetrachtung:

Bob $\approx 2^{20}$ Schritte (Erzeugung der Rätsel)
Senden $\approx 2^{20}$ Rätsel von Bob und Chiffirat von Alice
Alice $\approx 2^{20}$ Schritte (Brute-force-Angriff) und Verschlüsselung des Klartextes
Eve $\approx 2^{40}$ Schritte (Brute-force-Angriff)

Der Abhörer Eve muß im schlimmsten Fall 2^{20} Rätsel mit Brute-force entschlüsseln bis er x_0 gefunden hat und damit auch y_0 . Dazu braucht er aber ungefähr 2^{40} Schritte, da jedes Rätsel mit einem 20 bit langen Schlüssel verschlüsselt wurde. Es ist für Eve ungünstiger, das Chiffirat von Alice direkt mit einem Brute-force-Angriff zu entschlüsseln, da er dazu mehr als 2^{40} Schritte bräuchte, weil die Schlüssel y_i eine Länge von mehr als 40 bit haben.

Wenn Alice und Bob 10000 Operationen pro Sekunde ausführen können, dann benötigen sie jeweils ungefähr eine Minute. Eve muß aber bei vergleichbarer Rechenleistung etwa ein Jahr investieren. Man sieht also, daß der Aufwand für den Abhörer Eve unvergleichbar schneller wächst, wenn die Anzahl der Rätsel und die Länge der Schlüssel erhöht wird.

1.5 Einwegfunktionen

Einwegfunktionen bilden einen zentralen Baustein der Public-Key-Protokolle.

Definition 1.1 (Einwegfunktion) Eine *Einwegfunktion* ist eine injektive Funktion f , die einfach zu berechnen ist, deren Umkehrfunktion f^{-1} dagegen sehr schwierig zu berechnen ist.

Eine mathematisch exakte Definition wäre: Die Funktion f ist in polynomialer Zeit zu berechnen, die Umkehrfunktion f^{-1} aber nicht. Funktionen, die diese Eigenschaft erfüllen, sind bisher nicht bekannt.

Definition 1.2 (Einwegfunktion mit Falltür) Eine *Einwegfunktion mit Falltür* ist eine Einwegfunktion f , deren Umkehrfunktion f^{-1} einfach zu berechnen ist, falls man eine zusätzliche Information besitzt.

1.6 Hashfunktionen

Definition 1.3 (Hashfunktion) Eine *Hashfunktion* ist eine Funktion H , die eine beliebig lange Eingabe auf eine Ausgabe fester Länge (normalerweise kürzer) abbildet.

Beispiel 1.3 *Das XOR aller Eingabebytes ist eine Hashfunktion.*

Definition 1.4 (kryptographische Hashfunktion) Eine *kryptographische Hashfunktion* ist eine Hashfunktion, die zusätzlich noch folgende Bedingungen erfüllt:

- sie ist einfach zu berechnen,
- es ist schwierig zu einer vorgegebenen Ausgabe (Hashwert) ein Urbild (eine der höchstwahrscheinlich unendlich vielen Nachrichten mit diesem Hashwert) zu finden, und
- es ist schwierig zwei Nachrichten mit demselben Hashwert (Kollision) zu finden.

1.7 Hybrid-Kryptosystem

Public-Key-Kryptosysteme sind etwa einen Faktor 1000 langsamer als Blockchiffren bei gleichen Sicherheitsanforderungen. Bei den symmetrischen Verfahren muß aber vor der Kommunikation ein geheimer Schlüssel sicher ausgetauscht werden. Daher wird in den meisten praktisch eingesetzten Systemen zunächst ein geheimer Sitzungsschlüssel K mit einem Public-Key-Verfahren übermittelt. Danach wird ein schnelleres symmetrisches Verfahren verwendet, das dann den Sitzungsschlüssel K benutzt. Diese Systeme heißen *Hybridsysteme*, da zwei kryptographische Verfahren eingesetzt werden.

1. Bob schickt Alice seinen öffentlichen Schlüssel B .
2. Alice wählt einen Sitzungsschlüssel K , verschlüsselt ihn mit B und schickt $E_B(K)$ an Bob.
3. Bob entschlüsselt $D_{B'}(E_B(K)) = K$ mit seinem geheimen Schlüssel B' . Jetzt haben beide den gemeinsamen Sitzungsschlüssel K , der nur ihnen bekannt ist.
4. Ab jetzt läuft die Kommunikation mit einem schnellen symmetrischen Verfahren, das den Sitzungsschlüssel K verwendet.

1.8 Digitale Signatur

Eine digitale Signatur soll ähnlich wie eine handschriftliche Unterschrift folgende Eigenschaften erfüllen:

- authentisch, d.h. einer Person eindeutig zugeordnet;
- nicht kopierbar, d.h. Teile des Dokuments oder das Dokument sind nach der Unterschrift nicht änderbar;
- unabstreitbar;
- kurz im Vergleich zum Dokument.

Es ist schwierig, elektronisch alle Kriterien gleichzeitig zu realisieren.

1.8.1 Signatur mit Hilfe eines Notars und eines symmetrischen Kryptosystems

Übung: Geben Sie unter Verwendung eines symmetrischen Kryptosystems und eines Notars ein Signaturverfahren an. Wie werden die folgenden Aufgaben gelöst:

- Wie übermittelt Alice an Bob eine signierte Nachricht?
- Wie kann Bob Celia nachweisen, daß das Dokument von Alice signiert wurde?

Welche Kriterien werden von Ihrem Verfahren erfüllt?

1.8.2 Signatur mit Public-Key-Verfahren

Public-Key-Verfahren können auch zur Authentifizierung verwendet werden, indem man den geheimen Schlüssel zur Verschlüsselung benutzt.

1. Alice verschlüsselt die Nachricht m mit ihrem geheimen Schlüssel A' . Sie fügt dem Chiffre $c = D_{A'}(m)$ ihren Namen hinzu und schickt dies an Bob.
2. Bob entschlüsselt c mit Alices öffentlichen Schlüssel A und erhält die Nachricht $m = E_A(c)$.

Diese Signatur ist kopierbar. Diese Tatsache stört nicht bei der Unterschrift von Verträgen. Werden damit aber Schecks unterschrieben, so können diese mehrfach eingelöst werden. Es müssen Zeitstempel, laufende Nummern, etc. eingefügt und von der einlösenden Bank gespeichert werden, um dieses Problem zu beheben. Diese Unterschrift ist authentisch und unabstreitbar. Ihre Länge hängt von der Länge der Nachricht m ab. Sie ist also nicht kurz.

1.8.3 Signatur mit Hashfunktionen

Bei der Signatur mit Public-Key-Verfahren muß die ganze Nachricht m verschlüsselt werden. Dies ist oft mit hohem Rechenaufwand verbunden, wenn die Nachricht m groß ist. Dieses Problem läßt sich mit Hilfe einer Hashfunktion beheben.

1. Alice berechnet den Hashwert $h = H(m)$ der Nachricht m .
2. Alice verschlüsselt h mit ihrem geheimen Schlüssel A' und erhält ihre Unterschrift $D_{A'}(h)$ der Nachricht m .
3. Alice schickt ihre Nachricht m mit ihrer Unterschrift $D_{A'}(h)$ an Bob.
4. Bob berechnet den Hashwert der empfangenen Nachricht und vergleicht ihn mit $E_A(D_{A'}(h))$, wobei A der öffentliche Schlüssel von Alice ist.

Vorteil: Deutlich geringerer Rechenaufwand und Trennung von Nachricht und kurzer Unterschrift.

Nachteil: Es ist nicht so sicher. Die Sicherheit hängt von der Wirksamkeit der Angriffe gegen Hashfunktionen ab.

1.8.4 Angriffe gegen Hashfunktionen

Es soll zu einer Nachricht m eine weitere Nachricht m' gefunden werden, so daß $H(m) = H(m')$ (Kollision) gilt. Dies kann verwendet werden, um dem Empfänger eine falsche Nachricht zuzuspielen, ohne daß er dies bemerken kann.

- **Brute-force-Angriff**

Die Nachricht m und der Hashwert $H(m)$ seien bekannt. Es werden solange verschiedene Nachrichten m' ausprobiert, bis $H(m) = H(m')$ gilt. Durch Abändern der Nachricht m an n Stellen (durch Füllworte, Umstellungen) ergeben sich 2^n verschiedene Texte m' . Hat der Hashwert k bit Länge, so müssen etwa 2^k Nachrichten m' ausprobiert werden, bis $H(m) = H(m')$ auftritt.

- **Meet-in-middle-Angriff**

Man kann Hashfunktionen normalerweise derart modifizieren, daß sie eine Nachricht von hinten, also vom bekannten Hashwert $H(m)$ der Nachricht, rückhashen. Diese Tatsache kann ausgenutzt werden, um eine Kollision schneller als mit dem Brute-force-Angriff zu finden. Dazu wird die Nachricht m in zwei Teile m_1 und m_2 geteilt. Im ersten Teil werden Änderungen an $k/2$ Stellen vorgenommen. Es entstehen dadurch $2^{k/2}$ verschiedene Nachrichten m'_1 mit Vorwärtshashwerten $H(m'_1)$. Diese Nachrichten werden nach dem Hashwert sortiert.

Mit Hilfe des bekannten Hashwerts $H(m)$ wird solange der Rückwertshashwert $H_{rück}(m'_2, H(m))$ vom entsprechend veränderten zweiten Teil m'_2 der Nachricht m berechnet, bis eine Übereinstimmung des Rückwärtshashwerts $H_{rück}(m'_2, H(m))$ mit einem der Vorwärtshashwerte $H(m'_1)$ auftritt. Sie kann effizient mit Binärsuche festgestellt werden. Aufgrund des Geburtstagsparadoxon sind verhältnismäßig wenige Veränderungen an m_2 notwendig. Eine Kollisionsnachricht m' ergibt sich durch Hintereinanderschreiben von m'_1 und m'_2 .

Vergleich des Aufwands für $k=64$

- Brute-force-Angriff

Zeitaufwand $\approx 2^{64}$ Schritte (2^{64} m' hashen)

Speicheraufwand $\approx$ nur eine Nachricht

- Meet-in-middle-Angriff

vorwärts hashen $\approx 2^{32}$ Schritte (2^{32} m'_1 hashen)

Sortieren der Tabelle $\approx 32 \times 2^{32}$ Schritte

Speicheraufwand $\approx 2^{32} \times 8 \text{ byte} = 32 \text{ GByte}$ für $H(m'_1)$

Lösung des Meet-in-the-middle Problems: Will man k bit Sicherheit, so muß der Hashwert $2k$ bit lang sein.

1.9 Protokolle für Schlüsselaustausch

Es werden Protokolle benötigt, die den sicheren Austausch eines gemeinsamen Sitzungsschlüssels K zwischen zwei Teilnehmern koordinieren, um ihnen später den Einsatz eines schnellen symmetrischen Verfahrens zu ermöglichen.

1.9.1 Protokoll mit einem symmetrischen Verfahren und einer Schlüsselzentrale (SZ)

Die Schlüssel von Alice und Bob sind der SZ bekannt.

1. Alice teilt der SZ mit, daß sie mit Bob kommunizieren will.
2. Die SZ erzeugt einen Sitzungsschlüssel K , berechnet $E_A(K)$ und $E_B(K)$ und schickt dies an Alice.
3. Alice berechnet $D_A(E_A(K)) = K$.
4. Alice schickt $E_B(K)$ an Bob.
5. Bob berechnet $D_B(E_B(K)) = K$.
6. Alice und Bob besitzen jetzt einen gemeinsamen Sitzungsschlüssel K , der nur ihnen und der SZ bekannt ist.

Problem: Die SZ kennt die Schlüssel aller Teilnehmer und alle Sitzungsschlüssel und kann somit alle Nachricht abhören.

1.9.2 Protokoll mit Public-Key und einer Schlüsselzentrale

Die öffentlichen Schlüssel von Alice und Bob sind in einer Datenbank der Schlüsselzentrale gespeichert.

1. Alice holt Bobs öffentlichen Schlüssel B aus der Datenbank.
2. Alice erzeugt einen zufälligen Sitzungsschlüssel K , berechnet $E_B(K)$ und schickt dies mit ihrem Namen versehen an Bob.
3. Bob entschlüsselt $D_{B'}(E_B(K)) = K$ mit seinem geheimen Schlüssel B' .
4. Alice und Bob haben jetzt einen gemeinsamen Sitzungsschlüssel K .

Vorteil: Die SZ kennt nur die öffentlichen der Teilnehmer. Ihr sind die Sitzungsschlüssel nicht bekannt.

Dieses Protokoll bietet noch keine Sicherheit, da sich ein aktiver Angreifer in einem Man-in-the-middle Angriff zwischen die SZ und die Teilnehmer schalten kann. Er kann dann den Teilnehmer falsche Schlüssel schicken, ohne daß es von ihnen bemerkt wird.

1.9.3 Man-in-the-middle Angriff

Ein passiver Angreifer Eve kann nur das Public-Key-Verfahren oder das symmetrische Verfahren angreifen. Eve sollte keine Chance haben, diese Verfahren zu brechen. Ein aktiver Angreifer Mallory kann aber die Leitung unterbrechen und die Kommunikation, die über sie läuft, kontrollieren.

1. Alice schickt ihren öffentlichen Schlüssel A an Bob. Mallory fängt A ab und schickt seinen öffentlichen Schlüssel M an Bob mit Absender Alice.
2. Bob schickt seinen öffentlichen Schlüssel B an Alice. Mallory fängt B ab und schickt seinen öffentlichen Schlüssel M an Alice mit Absender Bob.
3. Alice verschlüsselt die Nachricht m und schickt das Chiffre $c = (E_M(m))$ an Bob.
4. Mallory entschlüsselt $D_M(c) = m$, liest die Nachricht und leitet $E_B(m)$ weiter an Bob.
5. Der Rückweg verläuft analog.

Die von Alice und Bob ausgetauschten Nachrichten können von Mallory nicht nur gelesen, sondern auch in beliebiger Weise manipuliert werden. Alice und Bob haben keine Möglichkeit festzustellen, ob ihre Nachrichten abgehört oder manipuliert wurden.

1.9.4 Interlock-Protokoll

Die besten Public-Key-Verfahren sind also wertlos, wenn die Teilnehmer falsche Schlüssel von einem Angreifer Mallory bei einem Man-in-the-middle-Angriff bekommen. Dieser Schwachpunkt kann teilweise durch das folgende Protokoll behoben werden.

1. Alice schickt Bob ihren öffentlichen Schlüssel A .
2. Bob schickt Alice seinen öffentlichen Schlüssel B .
3. Alice berechnet $E_B(m_A) = e_1 e_2$ und schickt die erste Hälfte e_1 an Bob.
4. Bob berechnet $E_A(m_B) = f_1 f_2$ und schickt die erste Hälfte f_1 an Alice.
5. Alice schickt die zweite Hälfte e_2 an Bob.

6. Bob berechnet $D_{B'}(e_1e_2) = m_A$. Bob beantwortet *sofort* die Frage von Alice und schickt die zweite Hälfte f_2 .
7. Alice beantwortet Bobs Frage $D_{A'}(f_1f_2)$.

Ein aktiver Angreifer Mallory kann bis zum 3-ten Schritt alles wie bisher machen. Dann muß aber Mallory eine neue Nachricht erfinden, sie verschlüsseln und die erste Hälfte an Bob schicken. Im 4-ten Schritt muß Mallory nochmals eine neue Nachricht erfinden, sie verschlüsseln und die erste Hälfte an Alice schicken. Im 5-ten Schritt weiß Mallory Alices Frage, muß aber die zweite Hälfte der erfundenen Nachricht an Bob schicken. Im 6-ten Schritt erwartet Alice eine Antwort von Bob, die jetzt aber Mallory beantworten muß. Mallory weiß entweder die Antwort oder muß raten. Der 7-ten Schritt verläuft analog. Durch das Interlock-Protokoll wird der Man-in-the-middle Angriff Mallory erschwert, jedoch nicht unmöglich gemacht.

Das Protokoll versagt, wenn Mallory die beiden Teilnehmer kennt oder wenn Alice und Bob sich nicht direkt kennen und daher keine geeigneten Fragen stellen können, d.h. kein gemeinsames Geheimnis besitzen. Das ist vor allem dann der Fall, wenn Alice und Bob keine Menschen, sondern Computer sind, die Fragen nur automatisch erzeugen können.

1.9.5 Schlüsselaustausch mit Signaturen

Das Interlockprotokoll erschwert einen Man-in-the-middle Angriff, macht ihn aber nicht unmöglich. Das Protokoll 1.9.2 kann mit Hilfe der digitalen Signatur so erweitert werden, daß ein Man-in-the-middle Angriff für einen Außenstehenden Mallory unmöglich wird.

Die Schlüsselzentrale unterschreibt die öffentlichen Schlüssel von Alice und Bob, wobei sie persönlich anwesend sein müssen. Diese von der Schlüsselzentrale signierten Schlüssel sind in einer Datenbank zugänglich. Jeder Teilnehmer kennt den öffentlichen Schlüssel der SZ und kann damit ihre Unterschrift prüfen. Mallory kann keinen Man-in-the-middle Angriff durchführen, da Alice und Bob die Authentizität der Schlüssel überprüfen können. Mallory kann jetzt nur noch zuhören oder die Leitung stören. Die SZ kann nicht wie bei der Vergabe von Sitzungsschlüsseln im Protokoll 1.9.1 alle Nachrichten abhören, ist also nicht mehr der große Schwachpunkt im System.

Die Schlüsselzentrale kann jetzt nur noch als einzige Instanz einen Man-in-the-middle Angriff durchführen.

Mit Hilfe einer SZ lassen sich auch sehr einfach Nachrichten verschicken. Der öffentliche Schlüssel von Bob wird von der SZ geliefert. Alice erzeugt Sitzungsschlüssel K , verschlüsselt Nachricht m mit symmetrischen Verfahren und schickt $(E_B(K), E_K(m))$ an Bob. Hier ist wieder wichtig, daß der öffentliche Schlüssel B signiert ist.

1.10 Zero Knowledge Protokolle

Bei einem Zero-knowledge Protokoll weist ein Beweiser B einem Verifizierer V nach, die Lösung eines bestimmten Problems P zu kennen. Bei diesem Nachweis wird keine Information über die Struktur der Lösung preisgegeben. Genauer ist gefordert, daß bei polynomial vielen Abläufen des Protokolls nicht mehr Information preisgegeben wird, als die, die der Verifizierer selbst mit polynomialen Aufwand hätte bekommen können.

Die Zero Knowledge Protokolle können für Authentifikationsverfahren benutzt werden.

Beispiel 1.4 *Der Keller hat eine Geheimtür. B will V beweisen, daß er die Geheimtür öffnen kann, ohne aber V das Geheimnis, wie sie geöffnet wird, zu verraten.*

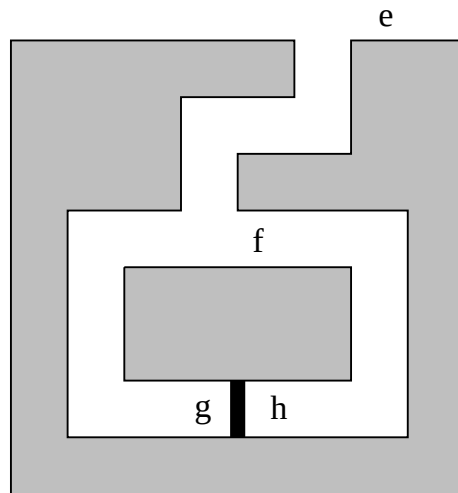


Abbildung 1.1: Keller mit Geheimtür

1. V steht bei e .
2. B geht entweder zu g oder h im Keller.
3. V geht zu f und ruft B zu, er solle links bzw. rechts herauskommen.
4. B öffnet bei Bedarf die Geheimtür und kommt am gewünschten Ort heraus.
5. Diese Prozedur wird n -mal wiederholt. Danach ist die Wahrscheinlichkeit, daß B in Wirklichkeit die Geheimtür gar nicht öffnen kann und jedesmal nur Glück hatte, nur $1/2^n$.

Definition 1.5 (Perfekt zero-knowledge) Ein Protokoll heißt *perfekt zero-knowledge*, wenn es einen Simulator gibt, der mit polynomialen Aufwand das Protokoll so simuliert, daß es identisch zur richtigen Protokollausführung ist. Der Simulator kennt dabei das Geheimnis nicht.

Die abstrakte Definition wird weiter unten eingehender erläutert. Es läßt sich auch eine etwas schwächere Definition erstellen, die die absolute Sicherheit durch *nicht berechenbar mit heutigen Mitteln* ersetzt:

Definition 1.6 (Berechenbar zero-knowledge) Ein Protokoll ist *berechenbar zero-knowledge*, wenn es einen Simulator gibt, der mit polynomialen Aufwand das Protokoll so simuliert, daß es mit heutigen Mitteln nicht von einer richtigen Protokollausführung zu unterscheiden ist.

Berechenbar zero-knowledge bietet zwar nicht den unbedingten Schutz von perfekt zero-knowledge, da durch neue Algorithmen oder schnellere Computer die Unterscheidung doch möglich werden kann. Es ist in vielen Fällen das einzige, was über ein Protokoll bewiesen werden kann.

Der Simulator ist das zentrale Instrument, mit dem die Zero-knowledge Eigenschaft definiert und bewiesen wird. Die Aufgabe dieses Simulators ist es, in Interaktionen mit einem echten Verifizierer die Rolle des Beweisers zu übernehmen. Dabei steht dem Simulator das Geheimnis des Beweisers nicht zur Verfügung. Gefordert ist nun, daß er es in einem nicht vernachlässigbaren (polynomial großen) Teil der Protokollausführungen schafft, die erwarteten Antworten zu geben und das Protokoll damit korrekt ablaufen zu lassen. Der Simulator stellt einerseits keinen erfolgreichen Angriff auf das Protokoll dar, da er bei konkreten Implementierungen nur in einem Teil der Fälle die richtigen Antworten geben kann. Andererseits sind die Antworten, in denen der Simulator korrekte Antworten geben konnte, nicht von den Antworten des Beweisers unterscheidbar. Daher kann sich ein Verifizierer mit Hilfe des Simulators beliebig (polynomial) viele Ausführungen des Protokolls beschaffen, ohne mit dem Beweiser interagieren zu müssen. Der Verifizierer erhält daher durch den Ablauf des Protokolls mit dem Beweiser keine Informationen, die er nicht auch mit dem Simulator hätte erhalten können. Deswegen kann der Beweiser sicher sein, daß er keine Information über sein Geheimnis preisgibt, wenn er das Protokoll polynomial oft ausführt.

Beispiel 1.5 *Der Keller mit Geheimtür ist perfekt zero-knowledge. Was würde passieren, wenn der Eingang keinen Knick machen würde?*

Ein Beispiel für einen Zero-knowledge Beweis wird am folgenden Protokoll, mit dem ein Beweiser nachweist, die Lösung des Rubik Würfels aus einer bestimmten Stellung zu kennen, gezeigt.

Beispiel 1.6 (Rubik Würfel) *B will V beweisen, daß er den Würfel aus einer bestimmten Stellung P lösen kann. Es wird davon ausgegangen, daß es schwierig ist, eine allgemeine Lösung des Würfels zu finden.*

1. *B transformiert den Würfel in eine andere Stellung P' mit Hilfe einer Transformation τ .*

2. *V entscheidet sich, ob er die Lösung aus der Stellung P' heraus oder die Transformation $\tau : P \leftrightarrow P'$ sehen will.*
3. *B führt das Verlangte vor.*

Ist der Beweiser echt, so kann er auf jede der beiden möglichen Fragen des Verifiziers eine korrekte Antwort geben. Ist er jedoch ein Betrüger, so kann er auf die Aufgabe „Löse den Würfel aus der Stellung P' “ nicht lösen, da er dafür eine allgemeine Lösung des Rubik Würfels kennen müßte, was aber nach der Grundannahme des Protokolls schwierig ist. Die Aufgabe „Zeige die Transformation $\tau : P \leftrightarrow P'$ “ kann er trivialerweise lösen.

Um zu ermitteln, ob das Protokoll zero-Knowledge ist, muß ein Simulator angegeben werden.

Der Simulator hat zwei Rubik Würfel, die sich im Anfangszustand bzw. im Zustand P befinden. Er rät zuerst die Herausforderung des Verifiziers. Nun kann er das Problem P' so konstruieren, daß er die Antwort kennt, falls sich der Verifizierer für die Herausforderung entscheidet, die er vorher geraten hat. Nachdem der Simulator nun das Problem P' präsentiert hat, fordert der Verifizierer den Simulator eine der Lösungen anzugeben. Stimmt die Herausforderung mit der geratenen überein, so gibt der Simulator die korrekte Antwort. Stimmen die Herausforderungen nicht überein, so gibt der Simulator auf, und fordert zu einem neuen Protokolldurchlauf auf.

Dieses Protokoll ist zwar perfekt zero-knowledge, aber der Verifizier kann aus dem Weg die Lösung erraten.

Aus diesem Beispiel läßt sich die typische Struktur eines kryptographisch nutzbaren Zero-knowledge Protokolls ableiten.

Typische Struktur eines zero-knowledge Protokolls:

Der Beweiser B will dem Verifizierer V beweisen, die Lösung eines Problems P aus einer Problemklasse $\mathcal{P}$ zu kennen:

1. B bestimmt ein zu P äquivalentes Problem $P' \in \mathcal{P}$ mit Hilfe einer Transformation $\tau : P \mapsto P'$ und sendet P' als Referenzwert an V .
2. V kann sich für eine Herausforderung (challenge) r entscheiden. Im Regelfall ist dies die Frage, ob er die Lösung des Problems P' oder die Transformation $\tau : P \mapsto P'$ sehen will.
3. B sendet die zum Referenzwert P' und der Herausforderung r passende Antwort.

Kennt der Beweiser die Herausforderung r bevor er seinen Referenzwert P' offenlegen muß, so braucht er die Lösung gar nicht zu kennen, da er sich zuerst die Antwort wählen kann und dann einen zur Herausforderung passende Referenzwert wählen kann. Daraus ergibt sich

die Struktur eines Simulators:

1. Wähle die Herausforderung r' zufällig und gleichverteilt.
2. Berechne einen zu r' passenden Referenzwert und eine dazugehörige Antwort.
3. Der Verifizierer stellt die Herausforderung r .
4. Falls $r \neq r'$ gehe zu Schritt 1.
5. Gebe den Protokollablauf mit Referenzwert, Herausforderung und Antwort aus.
6. Weiter mit Schritt 1.

Eine Möglichkeit für die Realisierung von Zero-Knowledge Protokollen ist relativ offensichtlich. Diese Möglichkeit basiert auf einem Beweiser, der behauptet eine Lösung einer Klasse von Problemen zu besitzen. Der Beweis kann dann einfach angetreten werden, indem der Verifizierer ein Problem aus dieser Klasse benennt und der Beweiser eine Lösung für dieses Problem vorlegt.

Beispiel 1.7 (Faktorisieren) *Der Beweiser behauptet, effizient große Zahlen faktorisieren zu können.*

1. V gibt eine große Zahl an.
2. B faktorisiert sie und V überprüft das Ergebnis.

Der Simulator S wählt sich einige Primzahlen und berechnet das Produkt r' . Falls der Verifizierer V eine Zahl r wählt mit $r = r'$, so kann S ihre Faktorisierung angeben. Das Problem ist es, daß die Wahrscheinlichkeit für $r = r'$ unendlich klein ist, da V jede der unendlich vielen Zahlen wählen kann. Gefordert ist aber, daß der Simulator in einem nicht vernachlässigbaren Teil der Protokollausführungen die erwarteten Antworten geben kann.

Beispiel 1.8 (Kenntnis von Quadratwurzeln) *Es sei n ein öffentlich bekannter, genügend großer Modulus. Die Faktorisierung sei nicht öffentlich bekannt. Das Berechnen von Wurzeln modulo n ist äquivalent zu der Faktorisierung von n und damit schwierig. Es sei y_B ein öffentlich bekannter Wert. Der Beweiser B behauptet, ein x_B mit $x_B^2 \equiv y_B \pmod{n}$ zu kennen. Das folgende Protokoll kann von ihnen benutzt werden, um diese Behauptung zu überprüfen.*

1. B wählt x zufällig und legt $y \equiv x^2 \pmod{n}$ offen.
2. V wählt sich $r \in \{0, 1\}$ und teilt es B mit.
3. B berechnet $z \equiv x_B^r x \pmod{n}$.
4. V überprüft ob $z^2 \equiv y_B y \pmod{n}$.

Durch die Multiplikation mit x wird das ursprüngliche Problem auf ein neues Problem, die Quadratwurzel aus $y_B y$ zu ziehen, transformiert. Falls V diese Quadratwurzel sehen will, kann B sie berechnen. Falls aber V die Transformation sehen will, so liefert V einfach die Zahl x . Die Transformation ist hier auch durch Wurzelziehen zu berechnen und deshalb genauso schwierig wie das ursprüngliche Problem.

Um einen Simulator zu konstruieren, halten wir uns an die oben beschriebene typische Struktur eines Simulators. Der Simulator wählt zufällig und gleichverteilt die Herausforderung $r' \in \{0, 1\}$. Falls der Verifizierer die andere Herausforderung $r \neq r'$ wählt, gibt der Simulator auf. Sonst kann er die richtigen Antworten geben:

Simulator S : $r' = 0$

1. S wählt x zufällig und legt $y \equiv x^2$ offen.
2. V entscheidet sich für $r = 0$.
3. S berechnet $z := x_B^0 x \equiv x$.
4. V überprüft, ob $z^2 \equiv y_B^0 y$.

Simulator S : $r' = 1$

1. S wählt x zufällig und legt $y \equiv y_B^{-1} x^2$ offen.
2. V entscheidet sich für $r = 1$.
3. S berechnet $z := x$.
4. V überprüft, ob $z^2 \equiv y_B^1 y$.

In der Hälfte der Fälle liefert der Simulator richtige Antworten, d.h. das Protokoll ist ein Zero-knowledge-Protokoll. Es ist sogar perfekt zero knowledge, da der Simulator nur quadratische Reste veröffentlicht.

Beispiel 1.9 (Graphenisomorphie) Der Beweiser kennt einen Isomorphismus ϕ zwischen den Graphen G_1 und G_2 und will dies dem Verifizierer beweisen. Dazu eignet sich folgendes Protokoll:

1. B transformiert den Graphen G_1 durch eine zufällige Permutation τ und erhält einen neuen Graphen $H = \tau(G_1)$.
2. V wählt, ob er $G_1 \mapsto H$ oder $G_2 \mapsto H$ sehen will.
3. B gibt den geforderten Isomorphismus an, da er die beiden Isomorphismen

$$\begin{aligned} \tau &: G_1 \longrightarrow H \\ \tau \circ \phi^{-1} &: G_2 \longrightarrow H \end{aligned}$$

kennt.

Das Protokoll ist perfekt zero-knowledge.

Kapitel 2

Public Key Algorithmen

Das Konzept der Public-Key-Kryptographie basiert auf der Tatsache, daß es möglich ist zwei Schlüssel (einen öffentlichen E zum Verschlüsseln und einen geheimen D zum Entschlüsseln) zu benutzen, wobei es aber praktisch unmöglich ist, den geheimen Schlüssel aus dem öffentlichen zu berechnen. Viele der vorgeschlagenen Public-Key-Algorithmen stellten sich als unsicher heraus und viele der sicheren Algorithmen sind nicht praktisch einsetzbar, da sie entweder zu große Schlüssel benötigen oder das Chiffre viel zu groß wird im Vergleich zu der Nachricht. Von den sicheren und praktisch einsetzbaren Algorithmen eignen sich nur einige für eine Schlüsselvergabe. Einige der Algorithmen sind nur zur Verschlüsselung und andere nur zur digitalen Signatur geeignet. Es gibt nur zwei Klassen von Algorithmen, die sich sowohl für Verschlüsselung als auch für digitale Signatur eignen und auch über längere Zeit häufig eingesetzt werden.

Die eine Klasse basiert auf dem Faktorisierungsproblem; das RSA-Verfahren ist der bekannteste Vertreter. Die andere Klasse basiert auf dem diskreten Logarithmusproblem in Gruppen; hier ist das Verfahren von ElGamal das bekannteste Verfahren. Diese Public-Key-Algorithmen sind langsam im Vergleich zu symmetrischen Verschlüsselungsalgorithmen. Daher werden in heutigen Systemen sowohl Public-Key-Komponenten, als auch symmetrische Komponenten eingesetzt; man spricht von Hybridkryptosystemen.

2.1 Knapsack

Die Sicherheit der Knapsack-Algorithmen beruht auf der Tatsache, daß das allgemeine Knapsackproblem ein NP-vollständiges Problem ist. Obwohl sich der ursprüngliche Algorithmus später als unsicher herausstellte, lohnt es sich dennoch, ihn zu untersuchen, um zu sehen, wie ein NP-vollständiges Problem für die Entwicklung eines Public-Key-Algorithmus verwendet werden kann.

Definition 2.1 (Das Knapsackproblem) Es seien beliebige Gewichte $a_1, \dots, a_n \in \mathbb{N}$ und eine beliebige Summe $s \in \mathbb{N}$ gegeben. Es wird eine Teilmenge von $\{a_1, \dots, a_n\}$ gesucht, so daß sich die in ihr enthaltenen Elemente zu s addieren. Anders formuliert lautet

das Problem: Finde ein $(x_1, \dots, x_n) \in \{0, 1\}^n$ mit $\sum_{i=1}^n x_i a_i = s$.

Obwohl das allgemeine Knapsackproblem NP-vollständig ist, gibt es spezielle Knapsäcke, die einfach zu lösen sind. Ein superincreasing Knapsack ist ein Beispiel dafür.

Definition 2.2 (Der superincreasing Knapsack) Es seien $b_1, \dots, b_n \in \mathbb{N}$ und $s \in \mathbb{N}$ ein Knapsack, wobei für alle i gilt $\sum_{j=1}^{i-1} b_j < b_i$.

Wegen dieser speziellen Struktur läßt sich der Knapsack $(b_1, \dots, b_n)$ mit einem Greedy-Algorithmus von oben her lösen.

Das nächste Beispiel zeigt einen sehr einfachen superincreasing Knapsack.

Beispiel 2.1 (Ein einfaches Knapsackproblem) Es seien $a_i = 2^{i-1}$ für $i = 1, \dots, n$. Damit ist es sehr einfach, das Knapsackproblem zu lösen. Der Lösungsvektor $(x_1, \dots, x_n)$ entspricht der rückwärts gelesenen Binärdarstellung von s .

Das Knapsackproblem kann wie folgt in einem Public-Key-Algorithmus verwendet werden:

- öffentlicher Schlüssel: $(a_1, \dots, a_n) \in \mathbb{N}^n$;
- geheimer Schlüssel: Ein Algorithmus, der den Knapsack $(a_1, \dots, a_n)$ in einen einfach lösbaren Knapsack transformiert;
- Verschlüsselung: $(x_1, \dots, x_n) \mapsto s = \sum_{i=1}^n x_i a_i$;
- Entschlüsselung: Löse das Knapsackproblem $(a_1, \dots, a_n)$ für die Summe s .

Merkle und Hellman stellten 1976 ein Verfahren vor, das die oben beschriebene Methode benutzt. Sie verwendeten einen superincreasing Knapsack und verschleierten ihn mit einer oder mehreren modularen Multiplikationen.

Verfahren:

- Wähle einen superincreasing Knapsack $(b_1, \dots, b_n)$ und verschleierte ihn durch eine lineare Transformation in einen anderen Knapsack. Wähle dazu $W, M \in \mathbb{N}$ mit $M > \sum_{i=1}^n b_i$ und $\text{ggT}(M, W) = 1$. Berechne

$$a_i := Wb_i \pmod{M} \quad (2.1)$$

und veröffentliche das Tupel $(a_1, \dots, a_n)$ als öffentlichen Schlüssel. Eine binäre Nachricht $(x_1, \dots, x_n)$ wird verschlüsselt zu $s := \sum_{i=1}^n x_i a_i$.

- Entschlüsselt wird durch Zurückführung auf das ursprüngliche Problem. Berechne $s' := W^{-1}s \pmod{M}$ (W^{-1} existiert, da $\text{ggT}(W, M) = 1$) und löse das Knapsackproblem $(b_1, \dots, b_n)$ für die Summe s' .

Angriff auf das Knapsackverfahren:

Das Knapsackverfahren weist folgende Schwachstellen auf:

- Es ist linear, d.h. es gilt $\sum_{i=1}^n (x_i + y_i) a_i = \sum_{i=1}^n x_i a_i + \sum_{i=1}^n y_i a_i$ und
- Wenn einige a_i ungerade sind, so ist ein Bit Information über den Klartext direkt zu sehen (s gerade oder ungerade).

Ein echter Angriff basiert auf der Beobachtung, daß aufgrund der Formel (2.1) Zahlen $k_1, \dots, k_n \in \mathbb{Z}$ mit

$$a_i U - k_i M = b_i \quad (2.2)$$

existieren, wobei $U \equiv W^{-1} \pmod{M}$. Dividiert man beide Seiten der Formel (2.2) durch $a_i M$, so erhält man

$$\frac{U}{M} - \frac{k_i}{a_i} = \frac{b_i}{a_i M}. \quad (2.3)$$

Weiterhin gilt $b_i \leq 2^{i-n} M$, da die Zweierpotenzen den kleinsten superincreasing Knapsack bilden. Subtrahiert man Gleichung (2.3) für $i := 1$ von der allgemeinen Gleichung (2.3), so ergibt sich

$$-\frac{k_i}{a_i} + \frac{k_1}{a_1} = \frac{b_i}{a_i M} - \frac{b_1}{a_1 M}$$

Multipliziert man diese Gleichung mit $a_1 a_i$, so erhält man

$$-a_1 k_i + a_i k_1 = \frac{a_1 b_i}{M} - \frac{a_i b_1}{M}. \quad (2.4)$$

Es gilt

$$|a_i k_1 - a_1 k_i| \leq \max \left\{ \frac{a_1 b_i}{M}, \frac{a_i b_1}{M} \right\} \quad (2.5)$$

$$\leq 2^{i-n} M. \quad (2.6)$$

Für sehr kleine i lassen sich damit Ungleichungen in $\mathbb{Z}$ angeben. Dabei ist a_i sehr groß, 2^{i-n} sehr klein. Es reichen nur wenige Ungleichungen, um die k_i eindeutig festzulegen. Mit dem Algorithmus von Lenstra zum linearen Programmieren lassen sich solche Ungleichungen in polynomialer Zeit lösen. Im Algorithmus „Faktorisieren von ganzzahligen Polynomen“ von Lenstra ist ein Teilalgorithmus „Finden von fast orthogonalen Basen in ganzzahligen Gittern“ enthalten. Dieser Algorithmus wurde 1983 implementiert und konnte auch zum Brechen des Knapsackverfahrens ausgenutzt werden. Eine zweite günstige Eigenschaft, die das Verfahren vollständig zusammen brechen läßt, ist es, daß nicht die ursprünglichen U, M gefunden werden müssen. Wenn ein $\frac{u}{m} \approx \frac{U}{M}$ verwendet wird, so entsteht ein äquivalenter Knapsack, der dieselbe Lösung $(x_1, \dots, x_n)$ besitzt.

2.2 RSA-Verfahren

Das RSA-Verfahren (benannt nach den Entdeckern Rivest, Shamir, Adleman) ist das bekannteste Public-Key-Verfahren. Die Sicherheit von RSA beruht auf der enormen Schwierigkeit, große Zahlen zu faktorisieren.

Verfahren:

1. Wähle zwei große Primzahlen p und q (70-200 Dezimalstellen oder mehr).
2. Berechne das Produkt $n = p \cdot q$.
3. Wähle einen öffentlichen Schlüssel e mit $\text{ggT}(e, (p-1)(q-1)) = 1$.
4. Berechne den geheimen Schlüssel d mit $e \cdot d \equiv 1 \pmod{(p-1)(q-1)}$ und veröffentliche das Tupel (n, e) als öffentlichen Schlüssel.

Mit Hilfe des Chinesische Restsatzes sieht man, daß $m^{ed} \equiv m \pmod{n}$ für alle $m \in \mathbb{Z}_n$ gilt.

- Verschlüsseln:
Es sind der Modulus n und der Schlüssel e öffentlich bekannt. Zur Verschlüsselung wird eine Nachricht m in $m_1, \dots, m_l$ zerlegt mit $m_i < n$, und es werden die Chiffre $c_i \equiv m_i^e \pmod{n}$ berechnet.
- Entschlüsseln:
Berechne $m_i \equiv c_i^d \equiv (m_i^e)^d \equiv m_i^{ed} \equiv m_i^1$ für $i = 1, \dots, l$ und setze daraus die Nachricht m zusammen.

Das RSA-Verfahren kann auch zum Signieren verwendet werden, indem man mit dem geheimen Schlüssel d verschlüsselt. Die Signatur kann durch Entschlüsseln mit dem öffentlichen Schlüssel überprüft werden.

Das RSA-Verfahren ist in Hardware etwa um einen Faktor 1000 und in Software etwa 100 mal langsamer als symmetrische Blockchiffren. Zur Beschleunigung wählt man geeignete einfache Exponenten (es wurden zum Beispiel $e = 3, 17$ und $2^{16} + 1$ in verschiedenen Standards vorgeschlagen). Es sind durch diese Einschränkung bisher keine größeren Probleme mit der Sicherheit bekannt.

Primzahlerzeugung:

Die Sicherheit des RSA-Verfahrens hängt in ganz entscheidender Weise davon ab, daß die verwendeten Primzahlen geheim bleiben. Deshalb müssen beim Erzeugen der Primzahlen nicht vorhersehbare Zahlen generiert werden; sie dürfen beispielsweise nicht mit einem Zufallsgenerator mit einem anderen Personen bekanntem Anfangszustand erzeugt werden.

Alle Verfahren, große Primzahlen zu erzeugen, wählen sich zunächst eine große Zufallszahl und testen diese auf Primalität. Bei den Primzahltests unterscheidet man zwischen zwei Klassen von Algorithmen:

- Probabilistische Primzahltests (relativ schnell und einfach),
- Deterministische Primzahltests (recht langsam und aufwendig).

Die wichtigsten Vertreter der probabilistischen Primzahltests werden im nächsten Kapitel vorgestellt.

Angriff auf Protokollebene:

Eve belauscht die von Bob an Alice geschickte Nachricht und kennt dadurch das Chiffriert $c \equiv m^e \pmod{n}$, wobei e der öffentliche und d der geheime Schlüssel von Alice sind. Eve wählt sich ein r zufällig mit $\text{ggT}(r, n) = 1$ und berechnet $x \equiv r^e \pmod{n}$ und $y \equiv xc \pmod{n}$. Eve schafft es weiterhin irgendwie, daß Alice die Nachricht y signiert (beispielsweise als Zeitstempelsignatur). Damit kennt Eve y^d . Sie berechnet

$$r^{-1}y^d \equiv r^{-1}x^d c^d \equiv r^{-1}(r^e)^d c^d \equiv r^{-1}rm \equiv m \pmod{n}$$

und erhält damit die Nachricht m . Alice führt eine blinde Signatur aus.

Angriff aufgrund eines gemeinsamen Modulus:

Jeder Teilnehmer erhält von der Zentrale sein Schlüsselpaar (e_i, d_i) , wobei für alle Teilnehmer der Modulus n gleich ist. Dadurch entsteht das folgende Problem: eine Nachricht m soll an zwei Personen geschickt werden. Es werden die beiden Chiffrierte m^{e_1} und m^{e_2} berechnet und an die entsprechenden Personen geschickt. Es gelte dabei $\text{ggT}(e_1, e_2) = 1$ für die geheimen Schlüssel der Personen. Wenn Eve die beiden Chiffrierte m^{e_1} und m^{e_2} belauscht, kann sie

$$re_1 + se_2 = 1$$

und damit auch

$$(m^{e_1})^r (m^{e_2})^s = m^{re_1 + se_2} = m^1$$

berechnen. Deswegen muß für jeden Teilnehmer ein eigenes Paar von Primzahlen p und q gewählt werden.

2.3 Diffie-Hellman-Verfahren

Das Verfahren von Diffie und Hellman erlaubt es, Alice und Bob ein gemeinsames Geheimnis über eine unsichere Leitung auszutauschen. Das Protokoll kann mit einer beliebigen endlichen zyklischen Gruppe realisiert werden. Die Gruppe G und ihr Erzeuger α sind öffentlich bekannt.

1. Alice wählt eine Zufallszahl a , berechnet α^a in G und sendet α^a an Bob.
2. Bob wählt eine Zufallszahl b , berechnet α^b in G und sendet α^b an Alice.
3. Alice empfängt α^b und berechnet $(\alpha^b)^a$.
4. Bob empfängt α^a und berechnet $(\alpha^a)^b$.

Alice und Bob haben jetzt ein gemeinsames Gruppenelement α^{ab} , das nur ihnen bekannt ist. Eve kennt durch Abhören nur die Gruppenelemente α^a und α^b . Es wird allgemein angenommen, daß die Gruppenelemente α^a und α^b nicht ausreichen, um α^{ab} auszurechnen, wenn es praktisch unmöglich ist, den diskreten Logarithmus von Gruppenelementen zu bestimmen.

Das *Diffie-Hellman-Problem* ist es, einen effizienten Algorithmus zu finden, der α^{ab} aus α^a und α^b berechnet. Es ist klar, daß die Lösung des diskreten Logarithmusproblem erlaubt, das Diffie-Hellman-Problem zu lösen. Die Umkehrung ist nicht bekannt.

2.4 ElGamal-Verfahren

Die Sicherheit des Verfahrens von ElGamal beruht auf der Schwierigkeit, den diskreten Logarithmus in endlichen zyklischen Gruppen zu bestimmen. Es kann sowohl zum Chiffrieren als auch zum Signieren von Nachrichten verwendet werden.

Sei α ein Erzeuger der Gruppe G . Die Gruppe G und der Erzeuger seien öffentlich bekannt. Die Nachrichten werden durch Gruppenelemente dargestellt. Alice will Bob eine Nachricht m senden. Bob wählt eine Zufallszahl x (sein geheimer Schlüssel), berechnet $y = \alpha^x$ und veröffentlicht das Gruppenelement y (sein öffentlicher Schlüssel). Für den Nachrichtenaustausch wird folgendes Protokoll verwendet:

1. Alice wählt eine Zufallszahl k und berechnet $a = \alpha^k$.
2. Alice holt sich Bobs öffentlichen Schlüssel y und berechnet $b = my^k$.
3. Alice sendet das Tupel (a, b) an Bob.

Bob kann die Nachricht m berechnen, da er seinen geheimen Schlüssel x kennt und a von Alice erhält.

Typischerweise werden die Gruppen $\mathbb{F}_{2^m}^*$, $\mathbb{F}_p^*$ (p Primzahl) oder die Gruppe $E(\mathbb{F})$ der Punkte auf einer elliptischen Kurve über einem endlichen Körper benutzt, da sie sich besonders einfach implementieren lassen. Die Gruppe $E(\mathbb{F})$ ist entweder eine zyklische Gruppe oder das direkte Produkt zweier zyklischer Gruppen. Normalerweise ist die eine Komponente des direkten Produkts eine kleine zyklische Gruppe und die andere eine große, die dann zum Verschlüsseln benutzt wird.

Ein Signaturverfahren läßt sich nur mit Primkörper realisieren. Betrachten wir den Fall $G = \mathbb{F}_p^*$ genauer. Bob wählt eine Primzahl p , ein primitives Element g und eine Zufallszahl

x . Er berechnet $y \equiv g^x \pmod{p}$. Sein öffentlicher Schlüssel ist das Tupel (y, g, p) . Sein geheimer Schlüssel ist der diskrete Logarithmus von y zur Basis g , die Zufallszahl x .

Chiffrierverfahren:

Alice verschlüsselt, eine Nachricht m , indem sie k zufällig mit $\text{ggT}(p-1, k) = 1$ wählt und $a \equiv g^k \pmod{p}$ und $b \equiv y^k m$ berechnet. Das Chifftrat ist das Tupel (a, b) . Bob kann das Chifftrat (a, b) entschlüsselt, indem er

$$m \equiv b(y^k)^{-1} \equiv b(g^{xk})^{-1} \equiv b((g^k)^x)^{-1} \equiv b(a^x)^{-1} \pmod{p}$$

mit seinem geheimen Schlüssel x ausrechnet.

Signaturverfahren:

Bob will eine Nachricht m signieren. Die Idee bei der Signatur ist es, die Nachricht m als

$$m \equiv ax \pmod{p-1}$$

auszudrücken. Die Signatur a kann dann durch

$$g^m \equiv g^{ax} \equiv y^a \pmod{p}$$

überprüft werden. Der geheime Schlüssel x kann aber aus der Signatur a

$$x \equiv ma^{-1} \pmod{p-1}$$

berechnet werden. Deswegen wird eine lineare Verschiebung benutzt. Dazu wählt Bob k zufällig mit $\text{ggT}(k, p-1) = 1$ und berechnet $a \equiv g^k \pmod{p}$. Bob berechnet außerdem noch ein b , so daß

$$m \equiv xa + kb \pmod{(p-1)}$$

gilt. Die Signatur ist dann das Tupel (a, b) . Andere Teilnehmer können Bobs Signatur überprüfen, indem sie

$$g^m \equiv g^{xa+kb} \equiv g^{xa} g^{kb} \equiv y^a a^b \pmod{p}$$

nachrechnen. Die Signatur funktioniert nur bei Primkörper, weil sonst Probleme mit dem Datentyp auftreten würden. Man müßt mit einem Gruppenelement potenzieren. Bei Primkörper werden die Gruppenelemente als natürliche Zahlen aufgefaßt. In den anderen Fälle ist dies nicht möglich.

Es ist wichtig, daß jedesmal k verschieden gewählt wird. Falls $a = g^k \pmod{p}$ zweimal gleich ist, kann das Linearegleichungssystem

$$\begin{aligned} m_1 &= xa + kb_1 \\ m_2 &= xa + kb_2 \end{aligned}$$

aufgestellt und der geheime Schlüssel x bestimmt werden.

2.5 McElice-Kryptosystem

Das McElice-Kryptosystem benutzt fehlerkorrigierende Goppa Codes, die einfach zu decodieren sind, und transformiert sie auf einen „allgemeinen“ linearen Code. Die Decodierung von allgemeinen linearen Codes ist ein NP-vollständiges Problem. Das McElice-System ist folgendermaßen aufgebaut:

- Geheimer Schlüssel
 - G' ist eine $k \times n$ -Matrix, die Generator-Matrix eines Goppa-Codes, der maximal t Fehler korrigieren kann;
 - P eine $n \times n$ -Permutationsmatrix;
 - S eine invertierbare $k \times k$ -Matrix.
- Öffentlicher Schlüssel
 - $G = SG'P$ eine $k \times n$ -Matrix;
 - die Nachricht ist ein Vektor der Länge k über $\{0, 1\}$;
 - verschlüsselt wird durch $c = mG + z$, wobei z ein zufälliger Vektor vom Gewicht ein wenig kleiner als t ist.
- Entschlüsseln
 - $c' = cP^{-1}$;
 - man decodiert m' , so daß $m'G' = c'$;
 - es gilt dann $m = m'S^{-1}$.

Der Vorschlag von McEliece war

- $n = 1024$,
- $t \approx 50$ Fehler,
- maximal mögliches k , für das ein Goppa-Code existiert, ist $k = 524$.

Bewertung

- Der Schlüssel hat etwa 2^{19} bit Länge.
- Die Nachrichtenlänge wird verdoppelt; der Rechenaufwand ist aber wegen der einfachen Verschlüsselung und Entschlüsselung sehr gering (deutlich schneller als RSA).
- Praktisch wird das Verfahren wegen der Schlüsselgröße kaum eingesetzt.

Angriffe

Direktes Decodieren braucht ungefähr 2^{500} Schritte. Dieser Aufwand läßt sich erheblich reduzieren, wenn man k Spalten der Matrix (mit der Hoffnung, daß dort kein Fehler dazuaddiert wurde) auswählt und das lineare Gleichungssystem $mM_k = c_k$ löst. Falls kein Fehler in diesen Spalten aufgetreten ist, kann man m durch Matrixinversion zurückgewinnen. Es müssen allerdings viele Matrizen getestet werden, bis bei 524 Spalten keiner der 50 Fehler enthalten ist. Der Aufwand beträgt etwa 2^{81} Schritte.

Kapitel 3

Mathematische Grundlagen

Dieses Kapitel ist stark an das Skript von Schaefer-Lorinser [4] angelehnt. Hier werden für das Verständnis der Public Key Kryptographie unbedingt notwendigen Teile aus [4] behandelt. Für ein besseres Verständnis der Primtests und Faktorisierungsalgorithmen ist die Lektüre des ausführlichen und weitergehenden Skripts wärmstens empfohlen.

3.1 Primzahlen

In diesem Kapitel werden wir uns mit ein paar grundlegenden Eigenschaften der Primzahlen beschäftigen, insbesondere werden wir ein paar Sätze kennenlernen, die die Verteilung der Primzahlen in den natürlichen Zahlen beschreiben. Beginnen wir jedoch mit der Definition von Primzahlen:

Definition 3.1 (Primzahl) Jede natürliche Zahl $n > 1$, die innerhalb der natürlichen Zahlen nur durch sich selbst und durch 1 teilbar ist, heißt *Primzahl*.

Die Primzahlen bilden die fundamentalen Atome der Arithmetik auf den natürlichen Zahlen, da sich jede natürliche Zahl in eindeutiger Weise als Produkt der Primzahlen darstellen läßt. Dies wird auch als der *Hauptsatz der Arithmetik* bezeichnet.

Um auf eine effiziente Weise eine Liste der Primzahlen kleiner einer vorgegebenen Zahl n zu erstellen, ist das auf den griechischen Mathematiker Eratosthenes (276-196 v.Chr.) zurückgehendes Verfahren „*Sieb des Eratosthenes*“ geeignet.

Schon aus der Antike ist der von Euklid ca. 325 v.Chr. in seinen Elementen notierte Satz bekannt:

Satz 3.1 (von Euklid) *Es gibt unendlich viele Primzahlen.*

Während uns dieser Satz also zeigt, daß in der Folge der natürlichen Zahlen immer wieder eine Primzahl vorkommt, zeigt der nächste Satz, daß auch beliebig große Lücken auftauchen können, das heißt Abschnitte der natürlichen Zahlen, die keine Primzahl enthalten.

Satz 3.2 *Zu jedem $k \in \mathbb{N}$ gibt es in den natürlichen Zahlen einen Abschnitt der Länge k , der keine Primzahl enthält.*

Beweis: Übung □

Das heißt also, die Primzahlen treten in gewisser Weise unregelmäßig auf. Es gibt keine geschlossene Formel, die beschreibt, welche natürlichen Zahlen Primzahlen sind. Dennoch kann eine recht präzise Aussage über die asymptotische Verteilung der Primzahlen gemacht werden, wie der folgende von Tschebyscheff bewiesene Satz zeigt. Wir definieren zunächst:

Definition 3.2 Es bezeichne für reelle Zahlen $x \in \mathbb{R}$ die Funktion $\pi(x)$ die Anzahl der Primzahlen, die nicht größer als x sind.

Satz 3.3 (Tschebyscheff) *Es gibt $a, b \in \mathbb{R}_{>0}$, so daß für $x \geq 2$ gilt*

$$a \frac{x}{\ln x} < \pi(x) < b \frac{x}{\ln x}. \quad (3.1)$$

Beweis: Übung □

Betrachten wir einen Restklassenring $\mathbb{Z}_n$ modulo einer natürlichen Zahl n , dann ist unmittelbar klar, daß dieser Ring nicht die Körperaxiome erfüllt, falls n keine Primzahl ist. Denn unter dieser Voraussetzung ist jeder echte Teiler von n nicht invertierbar. Die teilerfremden Zahlen a mit $1 \leq a \leq n$, für die $\text{ggT}(a, n) = 1$ gilt, sind in diesem Restklassenring $\mathbb{Z}_n$ invertierbar. Sie bilden die Einheitengruppe $\mathbb{Z}_n^*$, und die Anzahl der Elemente dieser Gruppe wird durch die sogenannte Eulersche Φ -Funktion angegeben.

Definition 3.3 (Eulersche Φ -Funktion) Die Funktion $\Phi(n)$ bezeichnet die Anzahl der natürlichen Zahlen kleiner n , die relativ prim zu n sind und wird die Eulersche Φ -Funktion genannt:

$$\#\mathbb{Z}_n^* = \Phi(n) = \#\{m \mid 1 \leq m < n, \text{ggT}(m, n) = 1\}$$

Wie kann diese Funktion explizit berechnet werden? Betrachten wir zunächst den einfachen Fall, daß n eine Primzahlpotenz p^e ist.

Bemerkung 3.4 (Primzahlpotenz p^e) *Es sei p eine Primzahl und $e \geq 1$. Dann gilt*

$$\Phi(p^e) = p^e - p^{e-1}.$$

Beweis: Für alle Produkte sp mit $s = 1, \dots, p^{e-1}$ gilt $\text{ggT}(sp, p^e) \neq 1$. Damit sind die $p^e - p^{e-1}$ restlichen Zahlen teilerfremd zu p^e . □

Es gilt ferner, daß die Eulersche Φ -Funktion für teilerfremde Zahlen multiplikativ ist:

Bemerkung 3.5 (Multiplikativität der Eulerschen Φ -Funktion) *Es seien $m, n \geq 1$ zwei natürliche Zahlen mit $\text{ggT}(m, n) = 1$. Dann gilt*

$$\Phi(m \cdot n) = \Phi(m) \cdot \Phi(n).$$

Beweis: Da m, n teilerfremd gilt $\text{ggT}(m \cdot n, s) = 1$ genau dann, wenn $\text{ggT}(m, s) = 1$ und $\text{ggT}(n, s) = 1$. Aufgrund der Isomorphie nach dem Chinesischen Restesatz

$$\mathbb{Z}_{mn} \cong \mathbb{Z}_m \times \mathbb{Z}_n$$

ergibt sich die Multiplikativität

$$\Phi(m \cdot n) = \Phi(m) \cdot \Phi(n)$$

unmittelbar, da

$$\mathbb{Z}_{mn}^* \cong \mathbb{Z}_m^* \times \mathbb{Z}_n^*.$$

□

Aus diesen Bemerkungen folgt unmittelbar das Korollar, das die Φ -Funktion für alle natürlichen Zahlen bestimmt.

Korollar 3.6 (Eulersche Φ -Funktion für beliebige Zahlen) *Jede natürliche Zahl n läßt sich eindeutig als $n = \prod_{i=1}^r p_i^{e_i}$ in seine Primfaktoren zerlegen. Damit gilt*

$$\Phi(p_1^{e_1} \cdots p_r^{e_r}) = \prod_{i=1}^r (p_i^{e_i} - p_i^{e_i-1}).$$

Der folgende Satz geht auf Fermat (1601-1665) zurück und wird oft als *der kleine Satz von Fermat* bezeichnet.

Satz 3.7 (der kleine Satz von Fermat) *Es sei p prim, dann gilt für alle $b \in \mathbb{Z}_p^*$*

$$b^{p-1} \equiv 1 \pmod{p}.$$

Das folgende weitergehende Resultat geht auf Euler (1707-1783) zurück. Es stellt eine Verallgemeinerung des kleinen Fermats dar, da es auch die Fälle, in denen p keine Primzahl ist, erfaßt.

Satz 3.8 *Es seien $n, b \in \mathbb{N}$ mit $\text{ggT}(b, n) = 1$, das heißt b ist eine Einheit in dem Ring $\mathbb{Z}_n$. Dann gilt*

$$b^{\Phi(n)} \equiv 1 \pmod{n}.$$

Da für n prim gilt: $\Phi(n) = n - 1$, ist Satz 3.7 ein Spezialfall davon.

3.2 Quadratische Reste

Wir wollen im folgenden die Frage untersuchen, wann sich eine Einheit a des Restklassenrings $\mathbb{Z}_n$ als Quadrat einer anderen Einheit x darstellen läßt.

Definition 3.4 (Quadratische Reste) Es seien $a, n \in \mathbb{N}$ mit $\text{ggT}(a, n) = 1$. Alle Elemente a , für die ein x existiert mit

$$x^2 \equiv a \pmod{n}$$

heißen *quadratische Reste* modulo n , die übrigen Einheiten heißen *quadratische Nicht-Reste*.

Für den Fall, daß es sich bei dem Modulus n um eine Primzahl handelt, definieren wir das *Legendre Symbol*:

Definition 3.5 (Legendre Symbol) Es sei p eine Primzahl und $a \in \mathbb{N}$. Dann heißt das durch

$$\left(\frac{a}{p}\right) := \begin{cases} 1, & \text{falls } a \text{ ein quadratischer Rest ist} \\ -1, & \text{falls } a \text{ kein quadratische Rest ist} \end{cases} \quad (3.2)$$

definierte Symbol $\left(\frac{a}{p}\right)$ das Legendre Symbol von a nach p .

Der folgende Satz gibt uns schon zwei wichtige Regeln für quadratische Reste bzw. quadratische Nicht-Reste.

Satz 3.9 *Das Produkt von zwei quadratischen Resten ist wieder ein quadratischer Rest, und das Produkt von einem quadratischen Rest und einem quadratischen Nicht-Rest ist ein quadratischer Nicht-Rest.*

Beweis: Der erste Teil des Beweises ist direkt einzusehen. Denn für zwei quadratische Reste a und b existieren x und y mit $x^2 \equiv a \pmod{n}$ und $y^2 \equiv b \pmod{n}$. Daraus folgt aber $(x \cdot y)^2 \equiv a \cdot b \pmod{n}$.

Den zweiten Fall beweisen wir durch einen Widerspruch. Angenommen, das Produkt von einem Quadratischen Rest x^2 und einem Quadratischen Nicht-Rest z wäre ein Quadratischer Rest y^2 . Daraus würde

$$x^2 \cdot z \equiv y^2 \pmod{n}$$

folgen. Da x als Einheit invertierbar ist, wäre dann z wegen

$$z \equiv (x^{-1} \cdot y)^2 \pmod{n}$$

ein Quadratischer Rest, was der Voraussetzung widerspricht. $\square$

Für den Fall eines Produkts von zwei quadratischen Nicht-Resten gibt es keine allgemeine Regel. Für den Fall aber, daß der Modulus n eine Primzahl ist, ergibt dies immer einen quadratischen Rest.

Satz 3.10 *Im Falle von quadratischen Resten modulo einer Primzahl p gilt:*

$$\left(\frac{a}{p}\right) \left(\frac{b}{p}\right) = \left(\frac{ab}{p}\right). \quad (3.3)$$

Ferner ist für $p \neq 2$ die Hälfte der Elemente der Einheitengruppe $\mathbb{Z}_p^$ Quadratische Reste und die andere Hälfte Quadratische Nicht-Reste.*

Beweis: Für $p = 2$ ist die Behauptung offensichtlich. Sei nun $p \neq 2$. Die Einheitengruppe $\mathbb{Z}_p^*$ ist eine zyklische Gruppe der Ordnung $p - 1$. Es gibt also einen Erzeuger g der Ordnung $p - 1$ und

$$g, g^2, \dots, g^{p-1}$$

ist bereits die ganze $\mathbb{Z}_p^*$. Zu jedem Element $a \in \mathbb{Z}_p^*$ existiert ein Index i mit

$$g^i \equiv a \pmod{p}.$$

Der Erzeuger g ist kein Quadratischer Rest, da sonst ein x mit $x^2 \equiv g \pmod{p}$ existieren würde. Nach dem Satz 3.7 würde für dieses x

$$x^{p-1} \equiv 1 \pmod{p}$$

gelten und somit $g^{(p-1)/2} \equiv 1 \pmod{p}$. Das ist aber ein Widerspruch dazu, daß g die ganze Einheitengruppe $\mathbb{Z}_p^*$ erzeugt. Offensichtlich sind alle geraden Potenzen von g Quadratische Reste

$$g^2, g^4, g^6, \dots, g^{p-1}$$

und da g ein Quadratischer Nicht-Rest ist, sind nach Satz 3.9 alle ungeraden Potenzen von g Quadratische Nicht-Reste

$$g, g^3, g^5, \dots, g^{p-2}.$$

Damit ist die Formel (3.3) bewiesen, da das Produkt von zwei ungeraden Potenzen eine gerade Potenz ergibt. $\square$

Auf Euler geht das folgende Ergebnis zurück, das uns ein Kriterium liefert, für ein gegebenes Element $a \in \mathbb{Z}_p^*$ direkt zu entscheiden, ob es ein quadratischer Rest oder nicht.

Satz 3.11 (Euler Kriterium) *Es sei $a \in \mathbb{N}$ mit $\text{ggT}(a, p) = 1$, wobei p eine ungerade Primzahl ist. Dann gilt*

$$\left(\frac{a}{p}\right) \equiv a^{\frac{p-1}{2}} \pmod{p}.$$

Beweis: Es sei g ein Erzeuger der Einheitengruppe $\mathbb{Z}_p^*$. Wir bestimmen zunächst ein $s \in \{0, \dots, p-2\}$ mit

$$g^s \equiv -1 \pmod{p}$$

. Für ein solches s gilt

$$g^{2s} \equiv (g^s)^2 \equiv (-1)^2 \equiv 1 \pmod{p}.$$

Dies kann nur gelten, wenn $2s$ ein Vielfaches der Gruppenordnung $\#\mathbb{Z}_p^* = p - 1$ ist, da $p - 1$ der kleinste Exponent e ist, für den $g^e \equiv 1 \pmod{p}$ gilt. Also kann s nur einen der Werte

$$0, \frac{p-1}{2}, p-1, \dots$$

annehmen. Es gilt wegen Satz 3.7

$$g^0 \equiv g^{p-1} \equiv 1 \pmod{p}.$$

Also muß $s = \frac{p-1}{2}$ sein und somit

$$g^{(p-1)/2} \equiv -1 \pmod{p}.$$

Da g ein Erzeuger der Einheitengruppe $\mathbb{Z}_p^*$ ist, kann jedes Element $a \in \mathbb{Z}_p^*$ als $a \equiv g^i \pmod{p}$ dargestellt werden. Damit gilt

$$a^{(p-1)/2} \equiv (g^i)^{(p-1)/2} \equiv \left(g^{(p-1)/2}\right)^i \equiv (-1)^i \pmod{p}.$$

Das heißt aber, daß $a^{(p-1)/2} \equiv 1 \pmod{p}$ genau dann gilt, wenn i ein gerader Exponent ist, und $a^{(p-1)/2} \equiv -1 \pmod{p}$ genau dann, wenn i ein ungerader Exponent ist. Dies ist aber das Kriterium für Quadratische Reste aus Satz 3.10. $\square$

Korollar 3.12 *Es sei p eine ungerade Primzahl und es gelte $ggT(a, p) = ggT(b, p) = 1$. Aus $a \equiv b \pmod{p}$ folgt, daß $\left(\frac{a}{p}\right) = \left(\frac{b}{p}\right)$.*

Beweis: Es ist $a \equiv b \pmod{p}$ gleichbedeutend damit, daß ein $l \in \mathbb{Z}$ existiert mit $b = a + l \cdot p$. Da modulo p gerechnet wird, gilt:

$$\begin{aligned} \left(\frac{b}{p}\right) &\equiv \left(\frac{a + l \cdot p}{p}\right) \\ &\equiv (a + l \cdot p)^{(p-1)/2} \\ &\equiv \sum_{i=0}^{p-1} \binom{p-1}{2i} a^{(p-1)/2-i} (lp)^i \\ &\equiv a^{(p-1)/2} \pmod{p}. \end{aligned}$$

$\square$

Ein sehr interessantes Ergebnis der Zahlentheorie, das wir ohne Beweis angeben, ist *das quadratische Reziprozitätsgesetz*, das von Gauß bewiesen wurde. Es stellt eine Relation zwischen dem Legendresymbol von zwei Primzahlen $\left(\frac{p}{q}\right)$ und der „Umkehrung“ $\left(\frac{q}{p}\right)$ dar.

Satz 3.13 (Quadratisches Reziprozitätsgesetz) *Seien p und q zwei ungerade Primzahlen. Dann gilt:*

$$\left(\frac{p}{q}\right) = \left(\frac{q}{p}\right) (-1)^{\frac{1}{2}(p-1)\frac{1}{2}(q-1)}. \quad (3.4)$$

Mit Hilfe des quadratischen Reziprozitätssatzes und des Korollars 3.12 läßt sich effizient entscheiden, ob eine gegebene Zahl a modulo einer Primzahl p ein Quadratischer Rest ist.

Beispiel 3.14 *Ist 19 ein Quadratischer Rest modulo 61? Es gilt*

$$\left(\frac{19}{61}\right) = \left(\frac{61}{19}\right) (-1)^{30 \cdot 9} = \left(\frac{61}{19}\right) = \left(\frac{4}{19}\right) = 1,$$

da $4 \equiv 2^2 \pmod{19}$.

Beim Legendre-Symbol sind Quadratische Reste modulo einer Primzahl p zugelassen. Im folgenden wird eine Verallgemeinerung des Legendre-Symbols, das Jacobi-Symbol, das auch für zusammengesetzte Zahlen definiert ist, eingeführt.

Definition 3.6 (Jacobi-Symbol) Es seien $a, n \in \mathbf{N}$ mit n ungerade und $\text{ggT}(a, n) = 1$. Die Primfaktorzerlegung von n sei gegeben durch $n = \prod_{i=1}^r p_i^{e_i}$. Dann ist das Jacobi-Symbol definiert durch

$$\left(\frac{a}{n}\right) := \prod_{i=1}^r \left(\frac{a}{p_i}\right)^{e_i}.$$

Das Jacobi-Symbol ist so definiert, daß es den gleichen Rechenregeln wie das Legendre-Symbol genügt. Es ist aber zu beachten, daß das Jacobi-Symbol $\left(\frac{a}{n}\right)$ *nicht* angibt, ob a ein quadratischer Rest modulo n ist.

Es ist leicht einzusehen, daß für ein a , das ein Quadratischer Rest modulo einer zusammengesetzten Zahl $n = \prod_{i=1}^r p_i^{e_i}$ ist, das Jacobi-Symbol $\left(\frac{a}{n}\right)$ den Wert 1 annimmt. Da a ein Quadratischer Rest modulo n ist, existiert ein $x \in \mathbf{N}$ mit $x^2 \equiv a \pmod{n}$. Betrachten wir diese Gleichung modulo p_i für $i = 1, \dots, r$, so erhalten wir $x^2 \equiv a \pmod{p_i}$. Es ist also a auch Quadratischer Rest modulo aller Primfaktoren p_i für $i = 1, \dots, r$ und damit ist das Jacobi-Symbol $\left(\frac{a}{n}\right)$ gleich 1.

Die Umkehrung dieser Aussage gilt nicht, wie das folgende Beispiel zeigt.

Beispiel 3.15 *Das Jacobi-Symbol kann auch für Quadratische Nicht-Reste den Wert 1 annehmen:*

$$\left(\frac{2}{15}\right) = \left(\frac{2}{3}\right) \left(\frac{2}{5}\right) = (-1)(-1) = 1,$$

obwohl 2 kein Quadratischer Rest modulo 15 ist.

3.3 Primtests

Die Primzahltests weisen eine ganze Reihe von Methoden und Parallelen zu den Faktorisierungsalgorithmen auf

3.3.1 Pseudoprimzahlen

Betrachten wir zunächst einen einfachen Satz, der ein Kriterium liefert, mit dem sich entscheiden läßt, ob eine Zahl n prim oder zusammengesetzt ist.

Satz 3.16 *Eine Zahl n ist prim genau dann, wenn*

$$a^{n-1} \equiv 1 \pmod{n}, \quad (3.5)$$

für alle $a \in \mathbb{N}$ mit $2 \leq a < n$.

Beweis: Die Hinrichtung gilt aufgrund des Satzes 3.7.

Für die Rückrichtung nehmen wir an, die Fermat-Bedingung (3.5) sei erfüllt und n sei zusammengesetzt. Dann hat n einen nichttrivialen Teiler r mit $1 < r < n$ und $n = r \cdot s$, und es gilt

$$r^{n-1} \equiv 1 \pmod{n}.$$

Es existiert also ein $l \in \mathbb{Z}$ mit

$$1 = r^{n-1} + l \cdot n = r^{n-1} + l \cdot s \cdot r = r^{n-1} + (l \cdot s) \cdot r.$$

Betrachten wir diese Gleichung modulo r , so ergibt sich

$$0 \equiv 1 \pmod{r}.$$

Das ist ein Widerspruch. □

Mit Hilfe dieses Satzes können wir bereits einen einfachen Primtest realisieren. Haben wir eine Zahl n gegeben, von der wir entscheiden möchte, ob sie prim oder zusammengesetzt ist, so wählen wir eine Basis a mit $2 \leq a < n$ und testen die Fermat-Bedingung (3.5). Erfüllt die Zahl n die Bedingung nicht, so ist n sicher eine zusammengesetzte Zahl. Andernfalls wählen wir eine neue Basis und führen den Test erneut durch.

Dies würde, falls n eine Primzahl ist, einen extrem aufwendigen Primtest ergeben. Wir können aber nach einer größeren Anzahl von Durchläufen des Primtests schon mit einer gewissen Wahrscheinlichkeit schließen, daß die Zahl n prim ist.

Definition 3.7 Eine zusammengesetzte Zahl n heißt *Pseudoprimzahl zur Basis a* , falls

$$a^{n-1} \equiv 1 \pmod{n}$$

erfüllt ist.

Beispiel 3.17 *Die Zahl $341 = 11 \cdot 31$ ist eine Pseudoprimzahl zur Basis 2, denn*

$$2^{341-1} \equiv 1 \pmod{341}.$$

Satz 3.18 *Es sei $n \in \mathbb{N}$ eine Pseudoprimzahl zur Basis a_1 und zur Basis a_2 mit $a_1, a_2 \in \mathbb{Z}_n^*$. Dann ist n auch eine Pseudoprimzahl zur Basis $a_1 \cdot a_2$ und zur Basis $a_1 \cdot a_2^{-1}$.*

Falls ein $a \in \mathbb{Z}_n^$ existiert, das die Bedingung*

$$a^{n-1} \equiv 1 \pmod{n}$$

nicht erfüllt, so wird sie auch von mindestens der Hälfte der Elemente aus $\mathbb{Z}_n^$ nicht erfüllt.*

Beweis: Der erste Teil ist durch direktes Nachrechnen zu beweisen. Es gilt

$$(a_1 \cdot a_2)^{n-1} \equiv a_1^{n-1} \cdot a_2^{n-1} \equiv 1 \pmod{n},$$

und damit ist n eine Pseudoprimzahl zur Basis $a_1 a_2$. Weiterhin gilt

$$(a_1 \cdot a_2^{-1})^{n-1} \equiv a_1^{n-1} \cdot (a_2^{n-1})^{-1} \equiv 1 \cdot 1^{-1} \equiv 1 \pmod{n},$$

und damit ist n eine Pseudoprimzahl zur Basis $a_1 a_2^{-1}$.

Es sei $B := \{a_1, \dots, a_l\}$ die Menge aller $a \in \mathbb{Z}_n^*$ mit $a^{n-1} \equiv 1 \pmod{n}$. Die zusammengesetzte Zahl n ist also eine Pseudoprimzahl zur Basis $a_i \in B$ für $i = 1, \dots, l$. Wir wählen $b \in \mathbb{Z}_n^* \setminus B$ beliebig aus. Wenn n eine Pseudoprimzahl zur Basis $b \cdot a_i$ wäre, dann wäre nach dem ersten Teil n auch eine Pseudoprimzahl zur Basis $(b \cdot a_i) a_i^{-1} \equiv b \pmod{n}$. Dies kann aufgrund der Wahl von b aber nicht sein, und damit sind alle Elemente aus $\{b \cdot a_1, \dots, b \cdot a_n\}$ Basen, zu denen n keine Pseudoprimzahl ist. Damit sieht man, daß mindestens die Hälfte der Elemente aus $\mathbb{Z}_n^*$ Basen sind, zu denen n keine Pseudoprimzahl ist. $\square$

Falls also zu einer zusammengesetzten Zahl n überhaupt eine Zahl $a \in \mathbb{Z}_n^*$ existiert, so daß n keine Pseudoprimzahl zur Basis a ist, so finden wir mit einer Wahrscheinlichkeit größer als $1/2$ eine solche Basis. Eine solche Basis a wird auch *Zeuge* für n genannt, weil die Angabe von a genügt, um zu beweisen, daß n eine zusammengesetzte Zahl ist.

Wählen wir nun k verschiedene Kandidaten für die Basis a auf zufällige Weise, so ist a für eine zusammengesetzte Zahl n nur mit Wahrscheinlichkeit kleiner als $1/2^k$ unter diesen Elementen kein Zeuge. Diese Aussage gilt wieder nur unter der Voraussetzung, daß es für n überhaupt einen Zeugen gibt.

Es gibt Zahlen, für die das nicht erfüllt ist. Sie werden *Carmichael-Zahlen* genannt. Wenn n eine Carmichael-Zahl ist, dann gilt

$$a^{n-1} \equiv 1 \pmod{n}$$

für alle $a \in \mathbb{Z}_n^*$.

Die Zahl $561 = 3 \cdot 11 \cdot 17$ ist beispielsweise eine Carmichael-Zahl.

Satz 3.19 *Es seien $m, n \in \mathbb{Z}$ ungerade. Für das Jacobi-Symbol gelten folgende Rechenregeln:*

(i)

$$\left(\frac{-1}{m}\right) = (-1)^{(m-1)/2},$$

(ii)

$$\left(\frac{2}{m}\right) = (-1)^{(m^2-1)/8},$$

(iii) Für m, n teilerfremd gilt:

$$\left(\frac{m}{n}\right) = \left(\frac{n}{m}\right) (-1)^{\frac{m-1}{2} \frac{n-1}{2}}.$$

3.3.2 Probabilistischer Primtest nach Solovay, Strassen

Im vorigen Abschnitt haben wir aus dem kleinen Fermat eine Art heuristischen Primtest abgeleitet, indem wir für die auf Primalität zu testende Zahl n überprüften, ob sie die Eigenschaft (3.5), die der kleine Fermat für alle Primzahlen liefert, erfüllt. Dieser einfache Primtest versagt mit sehr hoher Wahrscheinlichkeit bei den Carmichael-Zahlen. Wir wählen in dem Primtest eine Zufallszahl a mit $1 < a < n$. Ist $\text{ggT}(a, n) \neq 1$, so hat man einen Nullteiler gefunden, und weiß sofort, daß die Zahl n zusammengesetzt ist. Wie man aber leicht aus der Berechnungsvorschrift für die Eulersche Φ -Funktion sieht, ist die Menge der Nullteiler im Vergleich zu der Menge der Einheiten sehr klein. Ist n eine Carmichael-Zahl, so ist n eine Pseudoprimzahl zu allen Einheiten aus einheitn .

Die Grundidee des folgenden Primtests ist es, einen analogen Test bezüglich des Euler-Kriteriums vorzunehmen.

Definition 3.8 Analog zur Definition von Pseudoprimzahlen nennen wir zusammengesetzte Zahlen n , für die

$$\left(\frac{a}{n}\right) \equiv a^{(n-1)/2} \pmod{n}$$

gilt, *Euler-Pseudoprimzahlen* zur Basis a .

Das folgende einfache Lemma zeigt schon, daß dieser Primtest den auf dem kleinen Fermat basierenden Primtest umfaßt.

Lemma 3.20 Wenn n eine Euler-Pseudoprimzahl zur Basis a ist, dann ist n auch eine Pseudoprimzahl zur Basis a .

Beweis: Quadrieren der Gleichung im Satz 3.11 liefert die Behauptung. □

Der auf dieser Überlegung basierende Primtest wurde 1974 von Solovay und Strassen vorgeschlagen. Man nennt Algorithmen dieser Art probabilistisch, da es zusammengesetzte Zahlen gibt, für die der Test behauptet, die Zahl n sei mit einer gewissen Wahrscheinlichkeit prim. Durch Wiederholen des Primtests kann die Irrtumswahrscheinlichkeit exponentiell verringert werden. In der Praxis treten Zahlen, für die sich der Test auch nach mehrmaligem Anwenden irrt, äußerst selten auf.

Der Primzahltest nach Solovay und Strassen war das erste probabilistische Verfahren überhaupt. Heute werden in vielen Bereichen der Algorithmentechnik probabilistische Methoden eingesetzt.

Algorithmus:

1. Bestimme eine zufällig gewählte Zahl a mit $1 < a < n$.
2. Teste, ob $\left(\frac{a}{n}\right) \equiv a^{(n-1)/2} \pmod{n}$ gilt.
3. Falls ja, dann ist n wahrscheinlich prim, sonst ist n mit Sicherheit zusammengesetzt.

Das Jacobi-Symbol kann hierbei offensichtlich nicht über die Definition berechnet werden, da dies die Primfaktorzerlegung von n voraussetzt. Es läßt sich effizient mit den Rechenregeln ausrechnen.

Der große Vorteil dieses Primtests liegt nun darin, daß es für Euler-Pseudoprimzahlen nichts Äquivalentes zu den Carmichael-Zahlen gibt. Ferner läßt sich zeigen, daß die Irrtumswahrscheinlichkeit dieses Test kleiner als $1/2$ ist.

Satz 3.21 (Solovay, Strassen 1974) *Es sei $P := \{a \mid a \in \mathbb{Z}_n^*, \left(\frac{a}{n}\right) \equiv a^{(n-1)/2}\}$.*

- Falls n prim ist, so gilt $\#P = n - 1$.
- Falls n nicht prim ist, so gilt $\#P \leq \frac{1}{2}(n - 1)$.

Beweis: Falls n eine Primzahl ist, so liefert das Euler-Kriterium die Richtigkeit der Behauptung. Es sei nun n eine zusammengesetzte Zahl mit der Primfaktorzerlegung $n = \prod_{i=1}^r p_i^{e_i}$. Wir definieren zur Vereinfachung zwei Abbildungen

$$\epsilon : \begin{cases} \mathbb{Z}_n^* & \rightarrow \mathbb{Z}_n^* \\ a & \mapsto \left(\frac{a}{n}\right) \end{cases} \quad (3.6)$$

und

$$\delta : \begin{cases} \mathbb{Z}_n^* & \rightarrow \mathbb{Z}_n^* \\ a & \mapsto a^{(n-1)/2} \end{cases} \quad (3.7)$$

Es ist also zu zeigen, daß $\#\{a \in \mathbb{Z}_n^* \mid \epsilon(a) = \delta(a)\} \leq \frac{1}{2}(n - 1)$ erfüllt ist. Wir machen hierzu eine Fallunterscheidung.

1. Es gibt ein j mit $e_j > 1$ (n ist nicht quadratfrei).

Angenommen es sei $\epsilon(a) = \delta(a)$ für alle $a \in \mathbb{Z}_n^*$. Dann gilt

$$a^{n-1} \equiv \left(a^{(n-1)/2}\right)^2 \equiv \delta(a)^2 \equiv \epsilon(a)^2 \equiv \left(\frac{a}{n}\right)^2 \equiv 1 \pmod{n} \quad (3.8)$$

für alle $a \in \mathbb{Z}_n^*$. Es gilt ferner

$$\begin{aligned} \#\mathbb{Z}_n^* &= \Phi(n) \\ &= \Phi(p_1^{e_1}) \cdots \Phi(p_r^{e_r}) \\ &= p_1^{(e_1-1)}(p_1 - 1) \cdots p_r^{(e_r-1)}(p_r - 1). \end{aligned}$$

Daraus folgt $p_j \mid \#\mathbb{Z}_n^*$, da $e_j > 1$. Also gibt es ein Element $a \in \mathbb{Z}_n^*$ der Ordnung p_j , d.h. p_j ist der kleinste Exponent, für den $a^{p_j} \equiv 1 \pmod{n}$ gilt. Da p_j kein Teiler von $n-1$ ist, gilt $a^{n-1} \not\equiv 1 \pmod{n}$. Dies ist aber ein Widerspruch zu Gleichung (3.8). Es ist also $\epsilon \neq \delta$ und damit ist die Menge $\{a \in \mathbb{Z}_n^* \mid \epsilon(a) = \delta(a)\}$ eine echte Untergruppe (sie ist multiplikativ und enthält das neutrale Element) von $\mathbb{Z}_n^*$. Damit gilt für die Untergruppenordnung

$$\#\{a \in \mathbb{Z}_n^* \mid \epsilon(a) = \delta(a)\} \leq \frac{1}{2}\Phi(n)$$

und somit die Behauptung, da $\Phi(n) \leq n-1$ immer erfüllt ist.

2. Es gilt $e_i = 1$ für alle $i = 1, \dots, r$ ($\mathbb{Z}_n$ ist isomorph zu dem direkten Produkt der Primkörper $\mathbb{Z}_{p_1}, \dots, \mathbb{Z}_{p_r}$).

Angenommen, es sei $\epsilon(a) = \delta(a)$ für alle $a \in \mathbb{Z}_n^*$. Wähle a_1 mit $\left(\frac{a_1}{p_1}\right) = -1$ und a_i mit $\left(\frac{a_i}{p_i}\right) = 1$ für $i = 2, \dots, r$. Nach dem Chinesischen Restesatz existiert ein $a \in \mathbb{Z}$ mit $a \equiv a_i \pmod{p_i}$ für $i = 1, \dots, r$. Es gilt nach Definition des Jacobi-Symbols und Korollar 3.12

$$\epsilon(a) = \left(\frac{a}{n}\right) = \prod_{i=1}^r \left(\frac{a}{p_i}\right) = \prod_{i=1}^r \left(\frac{a_i}{p_i}\right) = -1 \cdot 1^{r-1} = -1$$

und nach der Annahme ist $\delta(a) = -1$. Betrachten wir den durch den Chinesischen Restesatz gegebenen Isomorphismus

$$\phi : \mathbb{Z}_n \longrightarrow \mathbb{Z}_{p_1} \times \dots \times \mathbb{Z}_{p_r}.$$

Da ϕ insbesondere ein Homomorphismus ist und aus $x \equiv -1 \pmod{n}$ für alle Primteiler p_i von die Gleichungen $x \equiv -1 \pmod{p_i}$ folgen, gilt speziell

$$-1 = \delta(a) = a^{(n-1)/2} \mapsto \left(a_1^{(n-1)/2}, \dots, a_r^{(n-1)/2}\right) = (-1, \dots, -1). \quad (3.9)$$

Nach dem Chinesischen Restesatz existiert ein $b \in \mathbb{Z}$ mit $b \equiv 1 \pmod{p_1}$ und $b \equiv a_i \pmod{p_i}$ für $i = 2, \dots, r$. Damit gilt

$$\delta(b) = b^{(n-1)/2} \mapsto \left(1^{(n-1)/2}, a_2^{(n-1)/2}, \dots, a_r^{(n-1)/2}\right) = (1, -1, \dots, -1).$$

Es kann also $b^{(n-1)/2} \equiv \pm 1$ auf keinen Fall gelten, da $b^{(n-1)/2}$ durch den Isomorphismus ϕ auf $(1, -1, \dots, -1)$ abgebildet wird und da aufgrund der Eigenschaft des neutralen Elements und der Formel (3.9) gilt

$$\begin{aligned} 1 &\mapsto \phi(1) = (1, 1, \dots, 1) \\ -1 &\mapsto \phi(-1) = (-1, -1, \dots, -1). \end{aligned}$$

Das Jacobi-Symbol $\left(\frac{b}{n}\right)$ kann nur die Werte 1 oder -1 annehmen. Es gilt aber $\left(\frac{b}{n}\right) = \epsilon(b) \neq \pm 1$. Also ist auch $\epsilon \neq \delta$ in diesem Fall gezeigt. Mit dem gleichen Untergruppenkriterium wie in ersten Fall folgt die Behauptung. $\square$

3.3.3 Probabilistischer Primtest nach Rabin und Miller

Der folgende Primtest wurde von Rabin aufbauend auf den Ideen von Miller entwickelt. Er liefert eine Verbesserung des Primtests von Solovay und Strassen, da die Menge der Fälle, in denen er sich irrt, eine echte Untermenge der Fälle ist, in denen sich der Primtest von Solovay und Strassen irrt.

Der Algorithmus geht von folgender grundsätzlicher Überlegung aus: Nehmen wir an, wir hätten eine Pseudoprimzahl n zur Basis a , also mit

$$a^{n-1} \equiv 1 \pmod{n},$$

gegeben. Betrachten wir Quadratwurzeln aus a^{n-1} , so würde für eine Primzahl n gelten, daß die Quadratwurzel ± 1 sind, da in einem Körper zwei Lösungen für $X^2 \equiv 1 \pmod{n}$ existieren. Das heißt, für eine Primzahl n würde die Folge

$$a^{(n-1)/2}, a^{(n-1)/2^2}, \dots, a^{(n-1)/2^k}$$

für $n-1 = 2^k m$ mit m ungerade aus entweder beliebig vielen 1en, gefolgt von einer -1 und dann beliebigen Zahlen aus $\mathbb{Z}_n^*$ oder lauter 1en bestehen. Diese Beobachtung wird in dem Miller-Rabin-Test ausgenutzt. Zur Berechnung geht man hier natürlich von $a^{(n-1)/2^k}$ aus und quadriert.

Algorithmus

1. Schreibe $n-1 = 2^k \cdot m$ mit m ungerade.
2. Wähle eine Zufallszahl a mit $1 < a < n$ und $\text{ggT}(a, n) = 1$.
3. Initialisiere $i := 0$ und $b := a^m$.
4. Falls $(i = 0 \text{ und } b = 1)$, dann ist n wahrscheinlich prim. Stop
5. Falls $b = -1$, dann ist n wahrscheinlich prim. Stop
6. Falls $(i > 0 \text{ und } b = 1)$, dann ist n sicher zusammengesetzt. Stop
Sonst $b := b^2$ und $i := i + 1$
7. Falls $i = k + 1$, dann ist n sicher zusammengesetzt. Stop
Sonst gehe zu Schritt 5.

Der folgende Satz liefert eine Abschätzung der Fehlerwahrscheinlichkeit dieses Algorithmus.

Satz 3.22 (Rabin) *Es sei $n \in \mathbb{N}$ ungerade und mit $n-1 = 2^k \cdot m$, wobei m ungerade ist. Zu $a \in \mathbb{N}$ mit $0 < a < n$ definieren wir eine Folge $a_0, \dots, a_k$ mit $a_i := a^{2^i \cdot m}$ für $0 \leq i \leq k$. Es sei*

$$P := \{a \in \mathbb{Z}_n^* \mid (a_0 \equiv 1 \pmod{n}) \text{ oder } (\exists i < k : a_i \equiv -1 \pmod{n})\}.$$

- Falls n prim, so gilt $\#P = n - 1$.
- Falls n nicht prim, so gilt $\#P \leq \frac{1}{4}(n - 1)$.

Beweis: siehe [4] □

Der Algorithmus erkennt die Primzahlen mit einer Wahrscheinlichkeit größer als $3/4$. Durch t -faches Wiederholen des Algorithmus kann die Fehlerwahrscheinlichkeit auf $1/4^t$ gesenkt werden.

Der nächste Satz zeigt, daß der Algorithmus von Rabin auch wirklich eine Verbesserung des Algorithmus von Solovay und Strassen darstellt.

Satz 3.23 *Es sei*

$$P' := \left\{ a \in \mathbb{Z}_n^* \mid \left(\frac{a}{n} \right) \equiv a^{(n-1)/2} \right\}$$

und

$$P := \{ a \in \mathbb{Z}_n^* \mid (a_0 \equiv 1 \pmod{n}) \text{ oder } (\exists i < k : a_i \equiv -1 \pmod{n}) \},$$

wobei $0 < a < n$, n ungerade und zusammengesetzt mit $n - 1 = 2^k \cdot m$ und $(a_i)_i$ eine Folge mit $a_0, \dots, a_k$. Dann gilt für zusammengesetzte Zahlen n : $P \subset P'$.

Beweis: siehe [4] □

3.4 Faktorisierungsalgorithmen mit Gruppen

Den beiden in diesem Abschnitt vorgestellten Algorithmen liegt eine gemeinsame Idee zugrunde. Der chinesische Restesatz besagt, daß für zusammengesetztes n der Restklassenring $\mathbb{Z}_n$ isomorph ist zu einem direkten Produkt von kleineren Restklassenringen. Wir definieren eine algebraische Struktur, die von dem Restklassenring $\mathbb{Z}_n$ abhängt. Wenn wir in dieser Struktur rechnen, so rechnen wir implizit auch in der algebraischen Struktur, die gewissermaßen von den kleineren Komponenten des direkten Produkts abhängt, und erhalten aufgrund des kleinen Fermats Information über eine der kleinere Strukturen und können dadurch einen Faktor von n abspalten. Im Falle der $p-1$ -Methode ist die algebraische Struktur $\mathbb{Z}_n$ selbst und bei der Faktorisierung mit elliptischen Kurven eine elliptische Kurve $E(\mathbb{Z}_n)$ über dem Restklassenring $\mathbb{Z}_n$.

3.4.1 Pollard $p-1$ -Methode

Der Aufwand dieses Algorithmus hängt im wesentlichen von der Größe des zu findenden Faktors ab. Nehmen wir also an, die Zahl n , die faktorisiert werden soll, enthält einen Primfaktor p , der die Eigenschaft besitzt, daß $p-1$ in kleine Faktoren zerfällt. Konkret sollen alle Faktoren von $p-1$ kleiner als eine Schranke k sein, und es soll darüberhinaus gelten, daß $k!$ durch $(p-1)$ teilbar ist.

Da die Exponentiation modulo n effizient berechnet werden kann, kann die Berechnung von

$$m := 2^{k!} \pmod{n}$$

für nicht zu große k und n durchgeführt werden. Der kleine Fermat liefert wegen $p-1 \mid k!$, daß

$$m \equiv 2^{k!} \equiv 2^{k! \pmod{p-1}} \equiv 2^0 \equiv 1 \pmod{p}$$

gilt. Damit teilt p also $m-1$. Falls k nicht zu groß angesetzt ist, teilt n mit einer hohen Wahrscheinlichkeit $m-1$ nicht, so daß der größte gemeinsame Teiler

$$g := \text{ggT}(m-1, n)$$

den nicht trivialen Teiler p von n liefert.

Bei der praktischen Anwendung dieser Vorgehensweise, weiß man natürlich nicht, in welcher Größenordnung der richtige Parameter k gewählt werden sollte. Damit nicht alle Primfaktoren von n erfaßt werden, ist es wichtig von Zeit zu Zeit $\text{ggT}(m-1, n)$ zu berechnen. Falls der ggT gleich 1 ist, so erhöht man k und rechnet weiter.

Sollte der ggT schon n sein, so sind alle Primfaktoren von n schon erreicht worden, so daß man entweder mit kleinerem k oder mit einer anderen Basis als 2 einen neuen Versuch macht.

Der Algorithmus führt nicht zum Erfolg, falls die Primfaktoren p_i von n eine ähnliche Zerlegung für p_i-1 haben. Dann gilt $(p_i-1) \mid k!$. In diesem Fall wird ganz n abgespalten.

Verfeinerung des Algorithmus

Anstatt $k!$ als Exponent zu verwenden, ist es günstiger eine etwas andere Wahl zu treffen, denn für sehr kleine Primzahlen q taucht q in $k!$ mit einem sehr hohem Exponenten auf. Es ist aber unwahrscheinlich, daß dies für die Bedingung $p-1 \mid k!$ benötigt wird. In der Praxis kann man beispielsweise so vorgehen, daß man bis zu einer gewissen Schranke B_1 , alle Primzahlen und Primpotenzen kleiner B_1 betrachtet. An Stelle von $k!$ wählt man dann das Produkt dieser Primzahlen und Primpotenzen. Das Produkt sei l .

Man wählt eine weitere Schranke $B_2 > B_1$ und testet zusätzlich zu

$$\text{ggT}(2^l - 1, n)$$

auch noch

$$\text{ggT}(2^{l^{q_i}} - 1, n)$$

für alle Primzahlen q_i mit $B_1 < q_i < B_2$, um die Primfaktoren zu trennen.

Laufzeitbetrachtungen

Die Laufzeit dieses Algorithmus hängt nicht nur von der Größe der zu faktorisierenden Zahl n ab, sondern wird auch stark von der Zerlegung von $p - 1$ für alle Primfaktoren p beeinflusst. Die Primfaktorzerlegung von n ist im voraus nicht bekannt. Daher kann man die Laufzeit des Algorithmus nur mit einer Wahrscheinlichkeitsbetrachtung abschätzen. Es sei p der kleinste Primfaktor von n . Die Laufzeit hängt dann nur von der größten Primpotenz von $p - 1$ ab, sofern p , wie im allgemeinen zu erwarten ist, als erster Faktor gefunden wird.

Betrachten wir $p - 1$ als zufällig gewählte natürliche Zahl, dann hat der größte Primfaktor im Mittel eine Größenordnung von $(p - 1)^{0.63}$. Falls also die größte Primpotenz kleiner der vorgegebenen Schranke B ist, so führt der Algorithmus zum Erfolg, wenn die Primfaktoren weit genug auseinander liegen.

3.4.2 Faktorisierung mit elliptischen Kurven

Der Ansatz, mit elliptischen Kurven zu faktorisieren, geht auf H.W. Lenstra zurück. Dieser Ansatz stellt in gewisser Hinsicht eine Verallgemeinerung der $p - 1$ -Methode von Pollard dar. Der Aufwand beim Faktorisieren mit elliptischen Kurven ist im wesentlichen auch von der Größenordnung des gesuchten Primfaktors abhängig.

Elliptische Kurven über Körpern

Es sei $\mathbb{K}$ ein Körper. Im Hinblick auf die betrachteten Anwendungen genügt es Körper $\mathbb{K}$ der Charakteristik $\text{char}(\mathbb{K}) \neq 2, 3$ zu betrachten. Dann lassen sich alle elliptischen Kurven beschreiben durch die Gleichungen der Form

$$y^2 = x^3 + Ax + B, \quad (3.10)$$

wobei $A, B \in \mathbb{K}$ mit $4A^3 + 27B^2 \neq 0$. Diese Nebenbedingung sichert, daß das Polynom $x^3 + Ax + B$ keine doppelten Nullstellen enthält.

Für $\mathbb{K} = \mathbb{R}$ lassen sich die elliptischen Kurven geometrisch veranschaulichen. Interessant werden die elliptischen Kurven $E(\mathbb{R})$ dadurch, daß man auf ihnen in einem abstrakten Sinne addieren kann. Zu je zwei Punkten P_1 und P_2 der Kurve findet man durch eine geometrische Konstruktion stets einen dritten, der ebenfalls auf der Kurve liegt und $P_1 + P_2$ genannt wird. Die so definierte Addition folgt den üblichen Rechenregeln: Sie ist assoziativ, kommutativ, es existiert ein neutrales Element, und zu jedem Punkt existiert ein Inverses. Eine elliptische Kurve bildet also eine kommutative Gruppe bezüglich dieser Addition.

Das Inverse $-P$ eines Kurvenpunktes P ist dessen Spiegelbild an der x -Achse. Die Summe zweier Punkte P_1 und P_2 findet man im allgemeinen, indem man die Gerade durch die Punkte P_1 und P_2 zieht. Sie schneidet die Kurve in genau einem dritten Punkt Q . Dessen Spiegelbild $-Q$ bezüglich der x -Achse ist die gesuchte Summe $P_1 + P_2$.

In dem Sonderfall $P_1 = P_2$ tritt an die Stelle der Geraden durch P_1 und P_2 die Tangente an die Kurve im Punkt $P_1 = P_2$. Wenn schließlich $P_1 = -P_2$ ist, trifft die Gerade durch P_1 und P_2 keinen weiteren Punkt der Kurve. Man stellt sich ersatzweise vor, daß die Gerade die Kurve im Unendlichen schneidet. Darum fügt man noch einen weiteren Punkt hinzu, den man sich im Unendlichen denkt. Man nennt ihn $\mathcal{O}$ und setzt $P_1 - P_1 = \mathcal{O}$. Der Punkt $\mathcal{O}$ ist das neutrale Element der additiven Gruppe $E(\mathbb{R})$.

Die geometrische Definition der Punktaddition kann in Formeln umgesetzt werden. Wenn $P_1 = (x_1, y_1)$ und $P_2 = (x_2, y_2)$ Punkte der elliptischen Kurve sind, dann gilt

$$-P_1 = (x_1, -y_1), \quad (3.11)$$

$$P_1 + P_2 = \begin{cases} \mathcal{O} & \text{falls } P_1 = -P_2 \\ P_2 & \text{falls } P_1 = \mathcal{O} \\ P_1 & \text{falls } P_2 = \mathcal{O} \end{cases} \quad (3.12)$$

In den anderen Fällen ist die Summe $P_3 = P_1 + P_2 = (x_3, y_3)$ definiert durch

$$\begin{aligned} x_3 &= \lambda^2 - x_1 - x_2 \\ y_3 &= \lambda(x_1 - x_3) - y_1 \end{aligned} \quad (3.13)$$

wobei

$$\lambda = \frac{y_1 - y_2}{x_1 - x_2}$$

falls $P_1 \neq P_2$, und

$$\lambda = \frac{3x_1^2 + A}{2y_1}$$

falls $P_1 = P_2$.

Diese Formeln können nun eine Art Eigenleben gewinnen. Man darf sie anwenden, ohne sich unter x und y Koordinaten von Punkten in der Ebene vorstellen zu müssen. Es kann ein beliebiger endlicher Körper $\mathbb{F}$ verwendet werden. Geraden und Tangenten haben dann nicht mehr viel mit geometrischer Anschauung zu tun.

Betrachten wir nun näher die elliptischen Kurven über Primkörpern $\mathbb{F}_p$. Offensichtlich ist $E(\mathbb{F}_p)$ endlich. Man schreibt üblicherweise die Anzahl der Elemente in der Form

$$\#E(\mathbb{F}_p) = p + 1 - t,$$

wobei t als die Spur bezeichnet wird. Für die Spur gilt die sogenannte Hasse-Schranke

$$-2\sqrt{p} < t < 2\sqrt{p}.$$

Die Gruppe $E(\mathbb{F}_p)$ ist entweder eine zyklische Gruppe oder das Produkt zweier zyklischer Gruppen.

Elliptische Kurven über Ringen

Die Definition einer elliptischen Kurve über einem Restklassenring ist ganz analog zur Definition der elliptischen Kurven über einem Körper.

Definition 3.9 Eine elliptische Kurve über dem Restklassenring $\mathbb{Z}_n$ ist definiert als die Menge der Paare $(x, y) \in \mathbb{Z}_n \times \mathbb{Z}_n$, die die Weierstrass-Gleichung

$$y^2 = x^3 + Ax + B$$

mit $A, B \in \mathbb{Z}_n$ und $4A^3 + 27B^2 \neq 0$ erfüllen zusammen mit einem Punkt im Unendlichen $\mathcal{O}$. Sie wird mit $E(\mathbb{Z}_n)$ bezeichnet. Wir setzen dabei voraus, daß 6 eine Einheit im Ring $\mathbb{Z}_n$ ist.

Auf dieser Punktmenge wird analog zur Addition auf elliptischen Kurven über Körpern eine Operation definiert. Eine direkte Verwendung der oben definierten Additionsformeln ist nicht möglich, da die dort auftretenden inversen Elemente nicht notwendigerweise existieren, da $\mathbb{Z}_n$ nicht nullteilerfrei ist, falls n nicht prim ist.

Es sei zur Vereinfachung n als quadratfrei vorausgesetzt, also $n = p_1 \cdot \dots \cdot p_r$ für paarweise verschiedene Primzahlen p_i für $i = 1, \dots, r$. Dann existieren die Abbildungen

$$h_i : E(\mathbb{Z}_n) \longrightarrow E(\mathbb{Z}_{p_i}),$$

die komponentenweise mittels der Projektionen

$$\pi_i : \mathbb{Z}_n \longrightarrow \mathbb{Z}_{p_i}$$

für $i = 1, \dots, r$ definiert sind. Durch Zusammensetzen dieser Abbildungen h_i erhält man eine Abbildung

$$h : E(\mathbb{Z}_n) \longrightarrow E(\mathbb{Z}_{p_1}) \times \dots \times E(\mathbb{Z}_{p_r}),$$

die den gesamten Zielraum ausschöpft, bis auf alle Tupel, in denen das neutrale Element einer der Kurven $E(\mathbb{Z}_{p_i})$ vorkommt und die nicht das Nulltupel sind. Dabei definieren wir die Abbildung h so, daß das neutrale Element $\mathcal{O}$ auf das Tupel bestehend aus den neutralen Elementen der Kurven $E(\mathbb{Z}_{p_1})$ bis $E(\mathbb{Z}_{p_r})$ abgebildet wird. Der Chinesische Restesatz liefert, daß allen Elementen im Wertebereich dieser Funktion ein Urbild in eindeutiger Weise zugeordnet werden kann. Für diese Fälle ist damit auch eine Operation auf $E(\mathbb{Z}_n)$ eindeutig definiert, in dem man die zu verknüpfenden Elemente zunächst mittels h nach $E(\mathbb{Z}_{p_1}) \times \dots \times E(\mathbb{Z}_{p_r})$ abbildet, dort koordinatenweise addiert und gegebenenfalls dem so erhaltenen Tupel wieder sein Urbild unter h zuordnet.

Die so definierte Operation kann auch durch Rechnen modulo n direkt in $\mathbb{Z}_n$ ausgeführt werden. Da diese Verknüpfung, falls sie definiert ist, der Addition auf elliptischen Kurven über Körpern entspricht, bezeichnet man sie als *Pseudoaddition*. Die mit dieser Operation versehene Kurve $E(\mathbb{Z}_n)$ bildet offensichtlich keine Gruppe, da die Operation nicht total definiert ist. Es gilt für jeden Punkt $P \in E(\mathbb{Z}_n)$

$$N_n \cdot P = \mathcal{O} \text{ mit } N_n := \text{kgV}(\#E(\mathbb{Z}_{p_1}), \dots, \#E(\mathbb{Z}_{p_r})).$$

Algorithmus

Ähnlich wie bei den Primtests führt man beim Faktorisieren mit elliptischen Kurven eine Berechnung aus, die gelingen würde, falls n eine Primzahl wäre. Weil n aber zusammengesetzt ist, kann die Rechnung scheitern. Im Verlauf der Berechnung muß man inverse Elemente berechnen, indem man den größten gemeinsamen Teiler von n mit anderen Zahlen ausrechnet. Die Rechnung kann nur dann fortgesetzt werden, wenn der größte gemeinsame Teiler gleich 1 ist, also die Zahlen teilerfremd sind. Falls der größte gemeinsame Teiler dann auch noch ungleich n ist, hat man einen echten Teiler von n gefunden.

Die Idee bei diesem Faktorisierungsalgorithmus ist es die Gruppe $\mathbb{Z}_n^*$, die bei der $p-1$ -Methode von Pollard verwendet wird, durch die allgemeinere Gruppe der rationalen Punkte über den elliptischen Kurven zu ersetzen. Auch hier ist der Algorithmus erfolgreich, falls $\#E(\mathbb{F}_p)$ in kleine Primfaktoren zerfällt. Der wesentliche Vorteil liegt jedoch darin, daß zu einem vorgegeben n verschiedene Gruppen mit verschiedener Ordnung aus dem Intervall

$$[p+1-2\sqrt{p}, p+1+2\sqrt{p}]$$

zur Verfügung stehen.

Für den Algorithmus kann man konkret wie folgt vorgehen. Zu einer Schranke b wählt man ähnlich wie bei der $p-1$ -Methode von Pollard einen Multiplikator $k := b!$ oder einen ähnlichen Multiplikator, der nur aus Faktoren kleiner b besteht. Dann wählt man sich eine elliptische Kurve $E(\mathbb{Z}_n)$ und einen Punkt P auf der Kurve. In der Praxis kann man dabei so vorgehen, daß man zunächst nur den Kurvenparameter A vorgibt, dann einen Punkt P zufällig wählt und dann den Kurvenparameter B so bestimmt, daß der Punkt P auf der Kurve liegt. Dann berechnet man $k \cdot P$ auf der Kurve. Diese Addition kann sehr effizient berechnet werden, da es nicht nötig ist k mal P zu addieren. Man kann ähnlich wie bei „square and multiply“ vorgehen. Die Addition führt man wie in der definierten Pseudoaddition durch. Bei der Berechnung müssen Inverse in $\mathbb{Z}_n$ bestimmt werden, die in $\mathbb{Z}_n$ nicht immer existieren müssen. Ein Nullteiler hat einen nichttrivialen Teiler mit n . In diesem Fall liefert die Berechnung des größten gemeinsamen Teilers einen Teiler von n .

Der Zufall spielt bei diesem Algorithmus wirklich mit, da es sehr viele elliptische Kurven zur Auswahl gibt, an denen man die Berechnungen durchführen kann. Lenstra konnte zeigen, daß es für jedes zusammengesetzte n Kurven gibt, die einen Teiler liefern. Falls man keinen nichttrivialen Teiler findet, kann man ein neues k bestimmen oder eine neue elliptische Kurve wählen.

Der Aufwand dieses Faktorisierungsalgorithmus läßt sich unter gewissen bisher unbewiesenen heuristischen Annahmen wie folgt abschätzen. Es sei p der kleinste Faktor der zu faktorisierenden Zahl n . Es sei ferner g eine beliebige natürliche Zahl. Dann findet der Algorithmus mit Wahrscheinlichkeit $\geq 1 - e^{-g}$ den Faktor p und hat einen asymptotischen Aufwand von

$$g \cdot \exp \left(\sqrt{(2 + o(1))(\log p \log \log p)} (\log n)^2 \right)$$

für $p \rightarrow \infty$.

Der Algorithmus eignet sich in hervorragender Weise zur Parallelisierung. Man wählt für jeden zur Verfügung stehenden Prozessor eine elliptische Kurve und rechnet auf allen Kurven solange parallel, bis ein Faktor gefunden wurde. Dabei ist nur minimale Vorbereitung und Nachbereitung auf jedem der Prozessoren notwendig.

3.5 Faktorisierung mit quadratischen Kongruenzen

Der Aufwand der Faktorisierungsalgorithmen mit Gruppen hängt von der Größe der Primfaktoren der zu faktorisierenden Zahl n ab. Für die Suche nach größeren Teilern benutzt man andere Algorithmen, die auf einer anderen Grundidee basieren. Die Laufzeit dieser Algorithmen hängt nur von n ab. Wir werden hier zwei Vertreter dieser Algorithmenklasse kennenlernen.

Man will eine zusammengesetzte Zahl n faktorisieren, indem man eine quadratische Kongruenz der Art

$$x^2 \equiv y^2 \pmod{n} \quad (3.14)$$

aufstellt. Diese Kongruenz ist gleichbedeutend mit der Aussage, daß n die Differenz $x^2 - y^2$ teilt. Nach der dritten binomischen Formel ist

$$x^2 - y^2 = (x - y)(x + y).$$

Wenn n nicht selbst Teiler von $x - y$ oder $x + y$ ist, so muß ein echter Teiler von n in $x - y$ und ein anderer echter Teiler in $x + y$ enthalten sein. Durch die Berechnung von $\text{ggT}(x + y, n)$ kann dann ein nicht trivialer Faktor gefunden werden.

Die Tatsache, daß sich über solche Kongruenzen ein Faktor finden läßt, ist schon länger bekannt. Sie läßt sich praktisch aber nur nutzen, wenn man geeignete Verfahren kennt, solche Kongruenzen zu finden. Dies wurde für nicht triviale Fälle erst durch die Verwendung von leistungsfähigen Computern möglich.

Zunächst werden wir den relativ einfachen Algorithmus von Dixon betrachten, an dem sich eine ganze Reihe von grundlegenden Ideen besonders gut erklären lassen. Er nimmt sofern eine Sonderstellung ein, als daß für diesen Algorithmus der subexponentielle Aufwand mit mathematischer Strenge bewiesen werden kann, während für die meisten anderen Varianten der Aufwand immer unter Verwendung unbewiesener, lediglich auf heuristische Annahmen gestützte Vermutungen abgeschätzt wird.

3.5.1 Grundlagen der Laufzeitanalyse

Bevor wir uns den eigentlichen Algorithmen zuwenden, führen wir einige Definitionen ein und stellen einige Sätze vor, die uns die Abschätzung der Laufzeit der Faktorisierungsalgorithmen erleichtern werden.

Wir benutzen im folgenden die Abkürzung

$$L(n) := \exp \left(\sqrt{\log n \log \log n} \right). \quad (3.15)$$

Ferner benutzen wir die Abkürzung L^α für die Funktionenklasse

$$L(n)^{\alpha+o(1)}. \quad (3.16)$$

Lemma 3.24 *Für diese Funktionenklasse gelten die folgenden Rechenregeln:*

- $2L^\alpha = L^\alpha$;
- $L^\alpha + L^\beta = L^{\max\{\alpha, \beta\}}$.

Beweis: Übung □

Lemma 3.25 *Für die Funktion $\pi(x)$, die die Anzahl der Primzahlen kleiner $x \in \mathbb{R}$ angibt, gilt*

$$\pi(L^\alpha) = L^\alpha \quad (3.17)$$

Beweis: Übung □

Eine wichtige Formel, die bei der Laufzeitanalyse benötigt wird, ist die Abschätzung der Anzahl der natürlichen Zahlen, die nur Primfaktoren kleiner einer vorgegebenen Schranke haben. Solche Zahlen werden als *glatt* (engl. *smooth*) bzw. *b-glatt* bezeichnet, wobei b die Schranke für die Größe der Primfaktoren ist. Dieser Begriff wurde von R.Rivest eingeführt.

Definition 3.10 Die Funktion $\Psi(x, y)$ gibt die Anzahl der Elemente $n < x$ an, die glatt bezüglich der Schranke y sind:

$$\Psi(x, y) := \#\{n \in \mathbb{N} \mid n < x \text{ und } ((p \mid n, p \text{ prim}) \Rightarrow p < y)\}. \quad (3.18)$$

Satz 3.26 *Für eine beliebige Konstante ϵ sei $D := \{(x, y) \mid x > 10 \text{ und } y \geq (\log x)^{1+\epsilon}\}$. Dann gilt für alle Paare $(x, y) \in D$*

$$\Psi(x, y) = x \cdot u^{-u+o(u)}$$

wobei $u := \frac{\log x}{\log y}$.

Lemma 3.27 *Für den für die Laufzeitanalyse wichtigen Spezialfall $\Psi(n, L^\alpha)$ gilt*

$$\Psi(n, L^\alpha) = n \cdot L^{-\frac{1}{2\alpha}}. \quad (3.19)$$

3.5.2 Dixons Algorithmus

Der Algorithmus von Dixon ist für praktische Zwecke nicht brauchbar, aber verschiedene Grundideen können anhand dieses Algorithmus besonders einfach erklärt werden. Außerdem handelt es sich um den historisch ersten Algorithmus, für den eine subexponentielle Laufzeit exakt nachgewiesen werden konnte.

Beschreibung des Algorithmus

Es sei n eine zusammengesetzte Zahl, die nicht durch Primzahlen $p < L(n)$ teilbar ist. Zu $m \in \mathbb{Z}$ bezeichne $Q(m)$ den kleinsten nicht-negativen Rest von $m^2 \pmod{n}$. Es sei a eine Konstante $0 < a < 1$, die wir später optimal festlegen werden. Um eine quadratische Kongruenz modulo n aufzustellen, benötigen wir den folgenden Teilalgorithmus:

1. Wähle $m \in \{1, \dots, n\}$ zufällig aus und berechne $Q(m)$.
2. Teste, ob $Q(m)$ nur Faktoren $p \leq L^a$ besitzt. Falls dies der Fall ist, so faktoriere $Q(m)$ vollständig und trage m , $Q(m)$ und das Tupel $(e_1, \dots, e_r)$, wobei $Q(m) = \prod_{i=1}^r p_i^{e_i}$ und $p_1, \dots, p_r$ die Primzahlen kleiner L^a sind, in einer Tabelle ein.

Dieser Teilalgorithmus wird so oft ausgeführt, bis $\pi(L^a) + 1$ Quadrate $Q(m)$, die L^a -glatt sind, gefunden wurden. Zu jedem Eintrag haben wir einen Vektor

$$v(m) \in \mathbb{F}_2^{\pi(L^a)},$$

dessen i -te Komponente eine 0 oder eine 1 ist, je nachdem, ob die Primzahl p_i in gerader oder in ungerader Potenz in $Q(m)$ auftritt.

Da wir $\pi(L^a) + 1$ viele Vektoren der Länge $\pi(L^a)$ haben, existiert eine lineare Abhängigkeit unter diesen Vektoren. Wir rechnen über dem Skalarkörper $\mathbb{F}_2$, das heißt uns interessiert nur, ob der Exponent gerade oder ungerade ist. Nach Umnummerierung können wir die lineare Abhängigkeit als

$$v(m_1) + v(m_2) + \dots + v(m_k) = \mathbf{0}.$$

ausdrücken. Es ist also das Produkt $Q(m_1) \cdot Q(m_2) \cdot \dots \cdot Q(m_k)$ ein Quadrat modulo n und es existiert folglich ein $x \in \mathbb{Z}$ mit

$$Q(m_1) \cdot Q(m_2) \cdot \dots \cdot Q(m_k) \equiv x^2 \pmod{n}.$$

Dieses x kann sehr einfach mit Hilfe der im 2-ten Schritt gespeicherten Tupel berechnet werden. Ferner läßt sich $y \equiv m_1 \cdot m_2 \cdot \dots \cdot m_k \pmod{n}$ berechnen und es gilt hierfür:

$$y^2 \equiv m_1^2 \cdot m_2^2 \cdot \dots \cdot m_k^2 \equiv Q(m_1) \cdot Q(m_2) \cdot \dots \cdot Q(m_k) \equiv x^2 \pmod{n}$$

Wir haben also eine quadratische Kongruenz modulo n aufgestellt. Durch Berechnen von $\text{ggT}(x + y, n)$ erhalten wir mit Wahrscheinlichkeit größer $1/2$ einen nichttrivialen Faktor von n . Falls $x \not\equiv \pm y \pmod{n}$ liefert die Berechnung von $\text{ggT}(x + y, n)$ sicher einen nichttrivialen Faktor von n . Das folgende Lemma besagt genauer:

Lemma 3.28 *Es seien $x, y \in \mathbb{Z}_n$ mit $x^2 \equiv y^2 \pmod{n}$ gegeben. Dann folgt daraus mit Wahrscheinlichkeit $1 - 1/2^{t-1}$, daß $x \not\equiv \pm y \pmod{n}$, wobei t die Anzahl der Faktoren von n ist.*

Beweis: Übung □

Bemerkung 3.29 *Man sieht also, daß bei dem RSA-Verfahren die Verwendung von mehr als zwei Primzahlen keine Sicherheitsvorteile bringt.*

Um die Erfolgswahrscheinlichkeit bei diesem Algorithmus zu erhöhen, berechnet man anstatt $\pi(L^a) + 1$ beispielsweise $\pi(L^a) + 10$ solche Einträge und erhält damit 10 verschiedene lineare Abhängigkeiten, von denen dann eine mit sehr hoher Wahrscheinlichkeit zu der gesuchten Zerlegung von n führt.

3.5.3 Laufzeitanalyse

Wir führen jetzt eine grobe Laufzeitanalyse durch, bei der auch der Parameter a optimal bestimmt wird.

In dem Teilalgorithmus müssen wir für $\pi(L^a) = L^a$ viele Primzahlen testen, ob sie $Q(m)$ teilen. Wir definieren $b := b(n)$ indirekt dadurch, daß wir sagen, wir benötigen L^b Durchläufe des Teilalgorithmus, bis wir $\pi(L^a) + 1$ viele geeignete Einträge gefunden haben. Dies ergibt einen Aufwand von L^{a+b} Schritten, bis wir genügend viele gute $Q(m)$ gefunden haben. Die Gaußelimination benötigt naiv implementiert L^{3a} Schritte. Die restlichen Berechnung der Werte x, y und $\text{ggT}(x + y, n)$ benötigt nur L^a viele Schritte. Also kann die gesamte Laufzeit des Algorithmus durch

$$L^{\max\{a+b, 3a\}}$$

abgeschätzt werden.

Wir müssen zur Bestimmung der Laufzeit die Funktion oder Konstante b ermitteln und a optimal wählen. Wir benutzen die heuristische Annahme, daß bei einer zufälligen Wahl von m auch das modulo n reduzierte Quadrat $Q(m)$ zufällig verteilt ist. Damit beträgt die Wahrscheinlichkeit, daß $Q(m)$ in Faktoren $p < L^a$ zerfällt,

$$\frac{\Psi(n, L^a)}{n},$$

da $\Psi(n, L^a)$ die Anzahl der L^a -glatte Zahlen kleiner n angibt und $\frac{1}{n}$ unter der heuristischen Annahme die Wahrscheinlichkeit jeder der möglichen Zahlen ist. Mit Lemma 3.27 sehen wir, daß

$$\frac{\Psi(n, L^a)}{n} = L^{-\frac{1}{2a}}$$

gilt. Um also $\pi(L^a) + 1 = L^a$ viele L^a -glatte $Q(m)$ zu finden, müssen wir

$$L^a \cdot \left(L^{-\frac{1}{2a}}\right)^{-1} = L^{a+\frac{1}{2a}}$$

viele Werte für m untersuchen. Also gilt für die Anzahl b der notwendigen Durchläufe des Teilalgorithmus

$$b = a + \frac{1}{2a},$$

und somit beträgt der Aufwand

$$L^{a+b} = L^{2a + \frac{1}{2a}}.$$

Die Funktion $2a + \frac{1}{2a}$ hat ihr Minimum bei $a = 1/2$. Dies ergibt eine Laufzeit für den gesamten Algorithmus von

$$L^{\max\{a+b, 3a\}} = L^{\max\{2a + \frac{1}{2a}, 3a\}} = L^{\max\{2, 1.5\}} = L^2,$$

das heißt es werden $L(n)^{2+o(1)}$ viele Schritte und ein Speicheraufwand von $L(n)^{1+o(1)}$ für die $L^a \times L^a$ -Matrix benötigt.

Satz 3.30 *Der Faktorisierungsalgorithmus von Dixon benötigt asymptotisch eine Laufzeit von $L(n)^{2+o(1)}$ vielen Schritten und einen Speicherplatzbedarf von $L(n)^{1+o(1)}$ bit.*

Unser Beweis dieses Satzes ist nicht mathematisch exakt durchgeführt worden, da wir eine heuristische Annahme benutzt haben. Diese Annahme betrifft die Wahrscheinlichkeit, mit der ein $Q(m)$ für ein zufällig gewähltes m in Primfaktoren $p < L^a$ zerfällt. Die $Q(m)$ sind nicht zufällig gleichverteilt, was wir angenommen haben. Es läßt sich jedoch zeigen, daß innerhalb einer Abweichung, die in $L(n)^{o(1)}$ abgefangen werden kann, dies auch für die Quadrate $Q(m)$ zutrifft.

3.5.4 Quadratischer Siebalgorithmus

Ein weiterer wichtiger Ansatz, wie eine Kongruenz der Art

$$x^2 \equiv y^2 \pmod{n}$$

aufgestellt werden kann, geht auf Pomerance zurück und verwendet ein ähnliches Verfahren, wie es beim Sieb des Eratosthenes eingesetzt wird. Da mit Hilfe eines solchen Sieb-Verfahrens eine Quadratzahl gefunden werden soll, spricht man von einem *quadratischen Sieb*.

In dem quadratischen Siebalgorithmus gehen wir von einem quadratischen Polynom

$$f(x) = (x + \lfloor \sqrt{n} \rfloor)^2 - n \tag{3.20}$$

aus. Für dieses quadratische Polynom mit ganzzahligen Koeffizienten gilt für beliebige $x \in \mathbb{Z}$

$$(x + \lfloor \sqrt{n} \rfloor)^2 \equiv f(x) \pmod{n}.$$

In dem quadratischen Siebalgorithmus betrachten wir statt $Q(m)$ die Werte $f(m)$, die in einer vorgegebenen Menge von B Primzahlen faktorisieren. Finden wir $m_1, \dots, m_k \in \mathbb{N}$, so daß

$$f(m_1) \cdot \dots \cdot f(m_k)$$

ein Quadrat ist, so erhalten wir mit

$$x^2 := f(m_1) \cdot \dots \cdot f(m_k)$$

und

$$y := (m_1 + \lfloor \sqrt{n} \rfloor) \cdot \dots \cdot (m_k + \lfloor \sqrt{n} \rfloor)$$

eine quadratische Kongruenz

$$x^2 \equiv y^2 \pmod{n},$$

mit der wir einen Faktor von n bestimmen können. Die Zahl x kann wie beim Dixon-Algorithmus bestimmt werden, indem man die gespeicherte Faktorisierung der $f(m_i)$ für $i = 1, \dots, k$ benutzt.

Wir haben hierbei nur eine gewisse Wahrscheinlichkeit, daß diese so gewonnene Gleichung wirklich zu einer Lösung führt, aber da wir mit geringfügigem Aufwand mehrere solche Gleichungen aufstellen können, läßt sich die Wahrscheinlichkeit einen Faktor zu finden, leicht erhöhen. Damit haben wir also das Problem, n zu faktorisieren, auf das Problem, k Zahlen $m_1, \dots, m_k$ zu finden, für die $f(m_1) \cdot \dots \cdot f(m_k)$ ein Quadrat modulo n ist, reduziert.

Wie groß sollte nun B gewählt werden? Eine geeignete Wahl für B ist sehr wichtig für den Algorithmus. Wenn wir B klein wählen, dann müssen wir weniger m_i testen, für die $f(m_i)$ in kleine Primfaktoren zerfällt, und das lineare Gleichungssystem kleiner. Andererseits wird durch diese Wahl das Finden einer Linearkombination des Nullvektors schwieriger. Ein geeignetes B stellt also einen Kompromis zwischen diesen gegenläufigen Aspekten dar.

Heuristische Überlegungen legen nahe, daß B in der Größenordnung von

$$\exp\left(\frac{1}{2}\sqrt{\log n \log \log n}\right)$$

gewählt werden sollte.

Nehmen wir nun an, daß wir $B + 10$ ganze Zahlen $m_1, \dots, m_{B+10}$ finden können, für die

$$f(m_i) = \prod_{j=1}^B p_j^{\alpha_{ji}}$$

über der Faktorbasis $\{p_1, \dots, p_B\}$ für $1 \leq i \leq B + 10$ faktorisieren. Die Exponenten α_{ji} sind dabei natürliche Zahlen.

Für jedes $i = 1, \dots, B + 10$ betrachten wir wie beim Dixon-Algorithmus den Vektor

$$v(m_i) = (\alpha_{1i}, \dots, \alpha_{Bi}) \in \mathbb{F}_2^B$$

Die Koeffizienten α_{ji} geben an, ob der Exponenten von p_j in $f(m_i)$ gerade oder ungerade ist. Wir haben also $B + 10$ Vektoren der Länge B , unter denen wir eine Linearkombination des Nullvektors suchen. Es existieren bis zu 10 verschiedene Lösungen, für die gilt

$$f(m_{i_1}) \cdot \dots \cdot f(m_{i_k}) = \prod_{j=1}^B p_j^{\alpha_{j,i_1} + \dots + \alpha_{j,i_k}}$$

und da

$$\alpha_{j,i_1} + \dots + \alpha_{j,i_k} \equiv 0 \pmod{2}$$

gilt, sind alle Exponenten gerade und somit ist

$$f(m_{i_1}) \cdot \dots \cdot f(m_{i_k})$$

ein Quadrat. Damit liefert dieses Vorgehen also 10 potentielle Lösungen, mit der eine quadratische Kongruenz aufgestellt werden kann.

Die Methode in der jetzigen Form eignet sich noch nicht zur Faktorisierung großer Zahlen. Je kleiner die $f(m)$ sind, um so größer ist die Wahrscheinlichkeit, daß $f(m)$ über einer vorgegebenen Faktorbasis $\{p_1, \dots, p_B\}$ zerfällt. Daher sollten wir möglichst kleine Werte für m nehmen, um geeignete $f(m)$ zu finden.

Aber selbst für m aus dem Intervall $1 \leq m \leq n^\epsilon$ mit $\epsilon > 0$ und klein, liegen die Werte von $f(m)$ wegen

$$f(m) = (m + \lfloor \sqrt{n} \rfloor)^2 - n = 2m\lfloor \sqrt{n} \rfloor + m^2 + (\lfloor \sqrt{n} \rfloor^2 - n) \approx 2m\lfloor \sqrt{n} \rfloor + m^2$$

in der Größenordnung $C\lfloor \sqrt{n} \rfloor$ für eine Konstante C , wenn $\epsilon < 1/4$ gilt.

Mit einem geeigneten Siebprozeß können wir auf wesentlich effektivere Weise geeignete Werte für m bestimmen. Wir konzentrieren uns dabei nicht auf einzelne Werte für m , sondern betrachten gleich eine ganze Reihe von Werten.

Dazu benötigen wir jedoch noch einige Vorüberlegungen. Zunächst ist klar, daß für einen Primfaktor p von $f(m_0)$ gilt, daß p auch Primfaktor von $f(m_0 + k \cdot p)$ für alle $k \in \mathbb{Z}$ ist. Um die Menge der Werte für m zu bestimmen, für die p ein Faktor von $f(m)$ ist, genügt es folglich, die quadratischen Kongruenzen

$$f(m) \equiv 0 \pmod{p}$$

für $m \in \{0, \dots, p-1\}$ zu lösen.

Aus der speziellen Form des quadratischen Polynoms (3.20) sieht man, daß für $p > 2$, das nicht selbst Primfaktor von n ist, die Kongruenz

$$f(m) = (m + \lfloor \sqrt{n} \rfloor)^2 - n \equiv 0 \pmod{p} \quad (3.21)$$

genau dann zwei verschiedene Lösungen besitzt, falls n ein Quadrat modulo p ist, d.h. das Legendresymbol $\left(\frac{n}{p}\right) = 1$. Falls n kein Quadrat modulo p ist, d.h. das Legendresymbol $\left(\frac{n}{p}\right) = -1$, existiert keine Lösung. In diesem Fall ist p kein Teiler von $f(x)$ für alle $x \in \mathbb{N}$. Wir bezeichnen daher mit $p_1, \dots, p_B$ nicht wie bei dem Dixon-Algorithmus die ersten B Primzahlen, sondern die ersten B Primzahlen p mit $\left(\frac{n}{p}\right) = 1$ inklusive 2. Diese Menge ist unsere *Faktorbasis*.

Die quadratische Siebprozedur funktioniert nun wie folgt. Zu jeder Primzahl aus der Faktorbasis werden die beiden Lösungen

$$\begin{aligned} a_p &\equiv n^{(p-1)/2} - \lfloor \sqrt{n} \rfloor \pmod{p} \text{ und} \\ b_p &\equiv -n^{(p-1)/2} - \lfloor \sqrt{n} \rfloor \pmod{p} \end{aligned}$$

Kongruenz (3.21) berechnet. Wir betrachten nun nicht nur einzelne zufällig gewählte Werte für m , sondern wir untersuchen ein Intervall von T vielen aufeinander folgenden m -Werten.

Zur Vorbereitung berechnen wir zunächst für jedes m den Wert $\lfloor \log_2 f(m) \rfloor$, also einfach die Anzahl bits, die für die Binärdarstellung von $f(m)$ benötigt werden. Dieser Wert ist für viele aufeinanderfolgende m -Werte gleich und daher effizient zu berechnen. Die m -Werte werden zusammen mit $\lfloor \log_2 f(m) \rfloor$ in einer Tabelle gespeichert.

Danach wird beim Sieben für jede Primzahl p aus der Faktorbasis der Wert $\lfloor \log_2 p \rfloor$ von dem Wert $\lfloor \log_2 f(m) \rfloor$ abgezogen, falls

$$m \equiv a_p \pmod{p} \text{ oder } m \equiv b_p \pmod{p}$$

gilt.

Für ein $f(m)$, das höhere Potenzen einer Primzahl p aus der Faktorbasis enthält, werden noch zusätzliche Tests benötigt. Das heißt für Primzahlen p mit $p^k < C$ muß für $i < k$

$$f(m) \equiv 0 \pmod{p^i}$$

getestet und gegebenenfalls mehrfach $\lfloor \log_2 p \rfloor$ abgezogen werden.

Am Ende des Siebens wurde also von jedem Eintrag $\lfloor \log_2 f(m) \rfloor$ für jeden Primteiler p von $f(m)$ der Wert $\lfloor \log_2 p \rfloor$ entsprechend oft abgezogen. Falls also $f(m)$ über der Faktorbasis zerfällt, ist dieser Eintrag innerhalb der Rechenungenauigkeit für die Rundung etwa gleich Null.

Mit diesem Sieb-Verfahren kann also aus einem Intervall von T vielen aufeinanderfolgenden Werten für m eine Liste von Werten $f(m)$ erstellt werden, die mit hoher Wahrscheinlichkeit über der Faktorbasis zerfallen. Diese Werte werden dann noch einmal im Einzelnen betrachtet, da während des Siebverfahren die Faktoren nicht protokolliert wurden. Aber jeder dieser Werte ergibt mit hoher Wahrscheinlichkeit einen Vektor über $\mathbb{F}_2$, so daß auf die oben skizzierte Art und Weise über die lineare Abhängigkeit ein Faktor von n gefunden werden kann.

Eine auf heuristische Annahmen gestützte Laufzeitanalyse ergibt für diesen Algorithmus einen Aufwand von

$$L(n)^{1+o(1)} = \exp \left((1 + o(1)) \sqrt{\log n \log \log n} \right).$$

Kapitel 4

Das diskrete Logarithmusproblem

Alle bis heute bekannten Algorithmen liefern keine effizienten Berechnungsverfahren zur Bestimmung des diskreten Logarithmus eines Gruppenelements aus einer endlichen zyklischen Gruppe. Damit wird das diskrete Logarithmusproblem neben dem Faktorisieren als eines der beiden bekannten harten Probleme angesehen, die sich für kryptographische Zwecke eignen. Das Verfahren von ElGamal stützt sich auf diese Tatsache.

4.1 Einführung

Es sei G eine endliche zyklische Gruppe und α ein Erzeuger von G . Dann gilt

$$G = \{\alpha^i \mid 0 \leq i < \#G\},$$

wobei $\#G$ die Gruppenordnung von G ist. Der diskrete Logarithmus eines Elements β zu der Basis α in G ist eine ganze Zahl x mit $\alpha^x = \beta$. Wenn x auf das Intervall $0 \leq x < \#G$ beschränkt ist, dann ist der diskrete Logarithmus von β zu der Basis α eindeutig. Wir schreiben $x = \log_\alpha \beta$.

Das diskrete Logarithmusproblem besteht darin, einen effizienten Algorithmus zu finden, der Logarithmen in einer gegebenen Gruppe berechnet.

Man kann sofort zwei Algorithmen zur Berechnung der diskreten Logarithmen in einer endlichen zyklischen Gruppe G angeben. Der erste berechnet einmal eine Liste der Logarithmen aller Gruppenelemente. Der zweite berechnet so lange Potenzen von α , bis eine Übereinstimmung mit β auftritt. Diese trivialen Algorithmen sind wertlos, wenn $\#G$ groß ist.

4.2 Baby-Step Giant-Step Algorithmus

Der Baby-Step Giant-Algorithmus eignet sich, um Logarithmen in beliebigen zyklischen Gruppen zu berechnen. Er stellt eine erhebliche Verbesserung gegenüber den in der

Einführung beschrieben trivialen Algorithmen, ist aber praktisch nicht einsetzbar, wenn die Gruppenordnung zu groß ist.

Wir wollen den diskreten Logarithmus $\log_\alpha \beta$ berechnen. Es sei $m = \lceil \sqrt{\#G} \rceil$.

1. Stelle eine Liste der Paare (i, α^i) für $0 \leq i < m$ (offensichtlich gilt $i = \log_\alpha \alpha^i$) und sortiere diese Liste nach der zweiten Komponente.
2. Berechne für jedes j mit $0 \leq j < m$ $\beta \alpha^{-jm}$ und überprüfe (mit Binärsuche), ob dieses Element als zweite Komponente eines Listeneintrags vorkommt.
3. Falls $\beta \alpha^{-jm} = \alpha^i$ für ein i mit $0 \leq i < m$, dann gilt $\beta = \alpha^{i+jm}$ und $\log_\alpha \beta = i + jm$.

Der Algorithmus stellt eine Liste mit $O(m)$ Einträgen und benötigt $O(m \log m)$ Operationen, um die Liste zu sortieren und dann in der Liste zu suchen. Mit einer Operation ist eine Gruppenoperation oder ein Vergleich gemeint. Der Name beschreibt in kennzeichnender Weise die Funktionsweise des Algorithmus.

Übung: Wie sieht die Giant-Step Baby-Step Version des Algorithmus?

4.3 Index Calculus

Wir beschreiben generisch die Funktionsweise der Index Calculus Methode. Es sei G eine endliche zyklische Gruppe der Ordnung n , die durch α erzeugt wird. Wir wollen die Logarithmen der Gruppenelemente zu dieser Basis berechnen. Es sei $S = \{p_1, p_2, \dots, p_t\}$ eine Untermenge von G mit der Eigenschaft, daß ein „bedeutender“ Teile der Gruppenelemente sich als Produkt der Elemente aus S schreiben läßt. Die Menge S wird als die *Faktorbasis* bezeichnet.

Im ersten Schritt versuchen wir die Logarithmen aller Elemente aus S zu bestimmen. Dazu wählen wir eine Zufallszahl a und versuchen α^a als Produkt der Elemente aus S zu schreiben:

$$\alpha^a = \prod_{i=1}^t p_i^{\lambda_i}. \quad (4.1)$$

Wenn wir so eine Darstellung finden können, erhalten wir aus (4.1) eine lineare Kongruenz

$$a \equiv \sum_{i=1}^t \lambda_i \log_\alpha p_i \pmod{n}. \quad (4.2)$$

Nachdem wir eine genügend große Anzahl (größer als t) an Relationen der Form (4.2) aufgestellt haben, können wir erwarten, daß das zugehörige lineare Gleichungssystem eine eindeutige Lösung für die Unbekannten $\log_\alpha p_i$ mit $1 \leq i \leq t$ besitzt.

Im zweiten Schritt berechnen wir individuelle Logarithmen in G . Es sei ein $\beta \in G$ gegeben, von dem wir den Logarithmus $x = \log_\alpha \beta$ bestimmen möchten. Wir wählen solange

Zufallszahlen s , bis $\alpha^s \beta$ sich als Produkt von Elementen aus S schreiben läßt:

$$\alpha^s \beta = \prod_{i=1}^t p_i^{b_i}. \quad (4.3)$$

Es gilt dann

$$\log_\alpha \beta \equiv \sum_{i=1}^t b_i \log_\alpha p_i - s \pmod{n}.$$

Um die Beschreibung der Index Calculus Methode abzuschließen, müssen wir angeben, wie ein geeignetes S zu wählen ist und wie effizient die Relationen (4.1) und (4.3) zu erzeugen sind. Ein geeignetes S ist klein (das Gleichungssystem im ersten Schritt ist dann nicht zu groß) und gleichzeitig muß die Anzahl der Gruppenelemente, die über S faktorisieren, groß sein (die erwartete Zeit, um die Relationen (4.1) und (4.3) aufzustellen, ist nicht zu lang). Solche Spezifikationen sind für einige Endliche Körper bekannt. Wir beschreiben kurz die Index Calculus Methode für endliche Körper. Die Algorithmen machen Gebrauch von den unterschiedlichen Darstellungsmöglichkeiten der Körperelemente. Deswegen wird eine kurze Einführung in die Theorie der endlichen Körper gegeben.

Die erwarteten Laufzeiten der Algorithmen werden durch die Funktion

$$L[x, \alpha, c] = O(\exp((c + o(1))(\log x)^\alpha)(\log \log x)^{1-\alpha}),$$

beschrieben, wobei x die Gruppenordnung ist, $0 \leq \alpha \leq 1$ und c Konstanten sind. Wenn $\alpha = 0$, dann ist $L[x, \alpha, c]$ polynomial in $\log x$. Bei $\alpha = 1$ ist $L[x, \alpha, c]$ dagegen exponentiell in $\log x$. Wenn $0 < \alpha < c$, dann ist $L[x, \alpha, c]$ *subexponentiell* in $\log x$.

4.3.1 Endliche Körper

Satz 4.1 (Primkörper) Für $m \in \mathbb{N}$ sind (i) m ist eine Primzahl und (ii) $\mathbb{Z}/m\mathbb{Z}$ ist ein Körper äquivalent.

Die endlichen Körper $\mathbb{F}_p = \mathbb{Z}/p\mathbb{Z}$, wobei p eine Primzahl ist, heißen *Primkörper*.

Definition 4.1 (Körpererweiterung) Unter einer Körpererweiterung versteht man ein Paar von Körpern $K \subset L$, wobei K ein Teilkörper von L ist. Bezeichnung: L/K oder $K < L$. L heißt Erweiterungskörper von K .

Es sei L eine Körpererweiterung von K . Man kann die Multiplikation auf L einschränken zu einer Multiplikation $K \times L \rightarrow L$ und auf diese Weise L als K -Vektorraum auffassen. Mit Hilfe dieser Beobachtung erhält man eine wichtige Strukturaussage über endliche Körper.

Es sei $\mathbb{F}_q$ ein endlicher Körper mit q Elementen und $\mathbb{F}_q/\mathbb{F}_p$. Dann ist $\mathbb{F}_q$ ein endlicher $\mathbb{F}_p$ -Vektorraum, da $\mathbb{F}_q$ endlich ist. Es sei $\dim_{\mathbb{F}_p}(\mathbb{F}_q) = [\mathbb{F}_q : \mathbb{F}_p] = n$ die Dimension und $\{\omega_1, \dots, \omega_n\}$ eine Basis von $\mathbb{F}_q$ über $\mathbb{F}_p$. Jedes Element von $\mathbb{F}_q$ läßt sich in eindeutiger

Weise als Linearkombination $a_1\omega_1 + \dots + a_n\omega_n$ mit $a_1, \dots, a_n \in \mathbb{F}_p$ schreiben. Die Anzahl der Elemente des Körpers $\mathbb{F}_q$ ist also p^n .

Durch die Vektorrauminterpretation erhalten wir die Strukturaussage, daß die Anzahl der Elemente eines endlichen Körpers eine Primzahlpotenz ist.

Definition 4.2 (Irreduzibel) Ein Polynom $f(x) \in K[x]$, $f \neq 0$, wobei K ein Körper ist, heißt *irreduzibel*, falls für jede Zerlegung $f(x) = g(x)h(x)$ mit $g(x), h(x) \in K[x]$ gilt $g(x) \in K^*$ oder $h(x) \in K^*$.

Satz 4.2 (Konstruktion eines Körpers mit p^n Elementen) *Es sei $f(x) \in \mathbb{F}_p[x]$ ein irreduzibles Polynom mit $\text{grad} f = n$. Dann ist der Restklassenring $\mathbb{F}_p[x]/f(x)$ sogar ein Körper.*

Beweis: Die Körpereigenschaft kann leicht dadurch verdeutlicht werden, indem der erweiterte ggT-Algorithmus ELBA betrachtet wird. Der ELBA stellt den ggT zweier Elemente eines euklidischen Rings als eine Linearkombination von ihnen dar. Wir fordern vorübergehend, daß $f(x)$ normiert ist, d.h. der Leitkoeffizient ist gleich 1. Da $f(x)$ ein normiertes irreduzibles Polynom vom Grad n ist, ist der $\text{ggT}(f(x), g(x)) = 1$ für alle Polynome $g(x) \in \mathbb{F}_p[x]$ mit $\text{grad}(g(x)) < n$. Der ELBA liefert die Darstellung

$$\text{ggT}(f(x), g(x)) = 1 = r(x)f(x) + s(x)g(x)$$

mit $r(x), s(x) \in \mathbb{F}_p[x]$. Wird die Berechnung modulo $f(x)$ ausgeführt, so ergibt sich direkt

$$1 \equiv s(x)g(x) \pmod{f(x)}$$

und $s(x)$ ist somit das inverse Element von $g(x)$ modulo $f(x)$. Die Eigenschaft, daß $f(x)$ normiert ist, ist unwesentlich, da wir sonst mit dem inversen Element des Leitkoeffizienten durchmultiplizieren können. Alle Element außer der Restklasse der Null haben bezüglich der Multiplikation ein Inverses. Der Restklassenring ist also ein Körper mit p^n Elementen, da so viele Restklassen modulo $f(x)$ existieren. $\square$

Satz 4.3 (Verfahren von Kronecker) *Es sei K ein Körper und $f(x) \in K[x]$ ein nicht-konstantes Polynom. Dann existiert ein Erweiterungskörper L ($K < L$), so daß $f(x)$ eine Nullstelle in L besitzt. Ist $f(x)$ irreduzibel, so kann man*

$$L := K[x]/f(x)$$

setzen.

Beweis: Es sei ohne Einschränkung $f(x)$ irreduzibel, ansonsten faktorisieren wir $f(x)$ und ersetzen $f(x)$ durch einen seiner irreduziblen Faktoren. Wir bilden die Komposition

$$\phi : K \xrightarrow{\iota} K[x] \xrightarrow{\pi} K[x]/f(x) =: L.$$

Der resultierende Homomorphismus $\phi = \pi \circ \iota$ ist injektiv. Wir können L als Erweiterungskörper von K auffassen, indem wir K mit seinem Bild $\phi(K)$ unter $\phi : K \rightarrow L$ identifizieren. Wir setzen $\bar{x} := \pi(x)$. Mit $f(X) = \sum_{i=0}^n c_i X^i$ gilt dann

$$\begin{aligned} f(\bar{x}) &= \sum_{i=0}^n c_i \bar{x}^i \\ &= \sum_{i=0}^n \pi(x)^i \\ &= \pi \left(\sum_{i=0}^n c_i x^i \right) \\ &= \pi(f(x)) \\ &= \bar{0}, \end{aligned}$$

d.h. $\bar{x}$ ist eine Nullstelle von $f(x) \in K[x]$ in L . $\square$

Bei dem Verfahren von Kronecker entsteht L aus K durch *Adjunktion einer Nullstelle* $\bar{x}$ von $f(x)$. Die Nullstelle $\bar{x}$ wird sozusagen erzwungen, indem wir ausgehend von dem Polynomring $K[x]$ den Restklassenring $L := K[x]/f(x)$ bilden. Wir können einen Linearfaktor von f über L abspalten und können das Verfahren wiederholen. Nach endlich vielen Schritten gelangt man zu einem Erweiterungskörper, über dem f vollständig in Linearfaktoren zerfällt.

Lemma 4.4 (Mehrfache Nullstellen) *Ein normiertes Polynom $f(x) \in K[x]$ hat genau dann keine mehrfachen Nullstellen, wenn $\gcd(f(x), f'(x)) = 1$.*

Definition 4.3 (Zerfällungskörper) Ein Zerfällungskörper eines Polynoms $f \in K[x]$ ist ein Erweiterungskörper L des Körpers K , in dem f vollständig in Linearfaktoren zerfällt.

Satz 4.5 (Isomorphie der minimalen Zerfällungskörper) *Es seien L_1 und L_2 zwei minimale Zerfällungskörper eines Polynoms $f \in K[x]$. Dann sind L_1 und L_2 isomorph.*

Lemma 4.6 (Gruppenordnung) *Es sei G eine Gruppe. Dann gilt für jedes Gruppenelement $g^{|G|} = 1$.*

Beweis: mit dem Satz von Lagrange. $\square$

Die multiplikative Gruppe $\mathbb{F}_q^*$ eines endlichen Körpers $\mathbb{F}_q$ hat die Ordnung $q - 1$, da alle Elemente außer des Nullelements invertierbar sind. Daher erfüllt jedes $\alpha \in \mathbb{F}_q$ die Gleichung $X^{q-1} = 1$. Wenn man noch das Nullelement berücksichtigt, erfüllt jedes Element $\alpha \in \mathbb{F}_q$ die Gleichung

$$f(x) = x(x^{q-1} - 1) = x^q - x = 0.$$

Das Polynom $f(x)$ hat q verschiedene Nullstellen in $\mathbb{F}_q$, nämlich die Elemente von $\mathbb{F}_q$. Das Polynom $f(x)$ zerfällt in Linearfaktoren über $\mathbb{F}_q$

$$f(x) = x^q - x = \prod_{\alpha \in \mathbb{F}_q} (x - \alpha).$$

Damit erhalten wir eine zweite Strukturaussage über endliche Körper. Alle Körper K mit $q = p^n$ Elementen sind isomorph, da sie minimale Zerfällungskörper des Polynoms $f(x) = x^q - x$ sind.

Diese Aussagen lassen sich zu folgendem Theorem zusammenfassen.

Theorem 4.7 (Struktur endlicher Körper) *Für jede Primzahl p und jedes $n \in \mathbb{N}$ existiert ein Körper $\mathbb{F}_{p^n}$ mit p^n Elementen. Er ist der minimale Zerfällungskörper des Polynoms $x^{p^n} - x$ und seine Elemente sind die Nullstellen dieses Polynoms.*

Jeder endliche Körper ist isomorph zu genau einem Körper $\mathbb{F}_{p^n}$.

Der folgende Satz ist für die Definition des diskreten Logarithmus in endlichen Körpern wichtig.

Satz 4.8 (Struktur der multiplikativen Gruppe $\mathbb{F}_q^*$) *Die multiplikative Gruppe eines endlichen Körpers ist zyklisch. Ein Element $\omega \in \mathbb{F}_q^*$ mit $\langle \omega \rangle = \mathbb{F}_q^*$ heißt primitives Element.*

4.3.2 Index-Calculus für $\mathbb{F}_p^*$

Wir stellen die Elemente von $\mathbb{F}_p^*$ als die Menge der Zahlen $\{1, 2, \dots, p-1\}$. Die Multiplikation wird modulo p ausgeführt. Der Erzeuger von $\mathbb{F}_p^*$ sei α . Es sei m eine Zahl, die eine Funktion von p ist. Die Faktorbasis S ist die Menge aller Primzahlen kleiner m .

Der erste und zweite Schritt werden wie im generischen Fall ausgeführt.

Wenn wir die Wahrscheinlichkeit, daß eine Zufallszahl kleiner p in Primfaktoren kleiner m zerfällt, betrachten, können wir m optimal wählen, um die Vorberechnungsphase (erster Schritt) und die Berechnung eines beliebigen Logarithmus (zweiter Schritt) zu beschleunigen. Das führt zu einer heuristischen aber subexponentiellen Implementierung des Index Calculus Algorithmus mit einer erwarteten Laufzeit von $L[p, 1/2, c]$. Die Laufzeit ist nicht rigoros, da angenommen wird, daß das Gleichungssystem im ersten Schritt vollen Rang hat.

4.3.3 Index-Calculus für $\mathbb{F}_{p^n}$

Die Elemente des endlichen Körpers $\mathbb{F}_{p^n}$ werden als Polynome vom Grad kleiner n aus $\mathbb{F}_p[x]$ aufgefaßt. Die Addition ist gewöhnliche Polynomaddition und die Multiplikation wird modulo eines festgelegten irreduziblen Polynoms $f(x) \in \mathbb{F}_p[x]$ ausgeführt. Wir benutzen also die Isomorphie $\mathbb{F}_{p^n} \cong \mathbb{F}_p[x]/(f(x))$.

Definition 4.4 Ein Polynom $w(x) \in \mathbb{F}_p[x]$ heißt *glatt* bezüglich der Schranke $b \in \mathbb{N}$, genau dann, wenn es in Polynome aus $\mathbb{F}_p[x]$ vom Grad kleiner b faktorisiert.

Mit der Polynomdarstellung des Körpers $\mathbb{F}_{p^n}$ ist es offensichtlich, die Faktorbasis S als die Menge aller irreduziblen b -glatten Polynome aus $\mathbb{F}_p[x]$ zu wählen. Im ersten Schritt werden die Logarithmen aller Elemente der Faktorbasis bestimmt. Es kann gezeigt werden, daß für kleine p und eine geeignet Wahl für b , der Index Calculus Algorithmus eine erwartete Laufzeit von $L[p^n, 1/2, c]$ hat.

Algorithmus:

Es sei $\alpha(x)$ ein Erzeuger von $\mathbb{F}_{p^n}^*$. Die Faktorbasis S wird als die Menge aller irreduziblen Polynome $p(x) \in \mathbb{F}_p[x]$ mit $\deg(p(x)) \leq b$ gewählt. Die Schranke b geben wir später an.

1. Vorberechnungsphase: es werden die diskreten Logarithmen aller Elemente aus der Faktorbasis S berechnet.

- (a) Wähle $a_i \in \{1, \dots, p^n - 1\}$ zufällig aus und berechne $y_i(x) := \alpha(x)^{a_i}$. Wir können effizient mit dem ggT-Algorithmus überprüfen, ob die Faktoren $p_i(x) \in S$ $y_i(x)$ teilen. Es werden alle maximalen Potenzen von $p_i(x) \in S$ von $y_i(x)$ abgespalten. Falls wir danach ein Polynom ungleich $c_i \in \mathbb{F}_p$ erhalten, wissen wir, daß $y_i(x)$ nicht b -glatt ist. Falls $y_i(x)$ b -glatt ist, so speichern wir die Faktorisierung

$$y_i(x) = c_i \cdot \prod_{p_j(x) \in S} p_j(x)^{e_{ij}}$$

mit $c_i \in \mathbb{F}_p$. Für die diskreten Logarithmen a_i gilt dann

$$a_i = \log_{\alpha(x)} y_i(x) = \log_{\alpha(x)} c_i + \sum_{p_j(x) \in S} e_{ij} \cdot \log_{\alpha(x)} p_j(x) \quad (4.4)$$

Wir wiederholen dies so lange, bis wir mehr als $\#S$ Gleichungen der Form (4.4) haben.

- (b) Löse dieses lineare Gleichungssystem mit den unbekannten $\log_{\alpha(x)} p_j(x)$ für $p_j(x) \in S$.
2. Berechnen eines konkreten Logarithmus von $y(x) = \alpha(x)^s$.

Wähle s so lange zufällig aus, bis $\alpha(x)^s y(x)$ b -glatt ist. Das Polynom $\alpha(x)^s y(x)$ kann dann wie folgt

$$\alpha(x)^s y(x) = c \cdot \prod_{p_j(x) \in S} p_j(x)^{e_j}$$

mit $c \in \mathbb{F}_p$ faktorisiert werden. Daraus kann bereits der diskrete Logarithmus $\log_{\alpha(x)} y(x)$ berechnet werden, denn es gilt

$$\log_{\alpha(x)} y(x) = \log_{\alpha(x)} c - s + \sum_{p_j(x) \in S} e_j \cdot \log_{\alpha(x)} p_j(x).$$

Die diskreten Logarithmen $\log_{\alpha(x)} p_j(x)$ für $p_j(x) \in S$ wurden im Schritt 1.(a) bestimmt und $\log_{\alpha(x)} c$ ist im (deutlich kleineren Grundkörper) einfach zu berechnen.

Die Schranke b sollte so gewählt werden, daß für die Menge S

$$\#S \approx \exp\left(c \cdot \sqrt{\log q \log \log q}\right)$$

gilt, wobei $q = p^n$.

4.4 Coppersmith-Algorithmus für $\mathbb{F}_{2^n}$

Der Coppersmith-Algorithmus basiert auf der speziellen Darstellung des Körpers $\mathbb{F}_{2^n} \cong \mathbb{F}_2[x]/(x^n + f_1(x))$ mit $\deg(f_1(x)) \leq \log_2(n)$. Ein solches Polynom $f_1(x)$ existiert für genügend große n .

1. Erzeugung der Datenbasis

Es werden die Logarithmen der Elemente aus der Menge S berechnet, wobei S die irreduziblen Polynome vom Grad kleiner s enthält. Dabei wird $s \approx (n \log^2(n))^{1/3}$ und das primitive Element $\alpha(x) \in S$ gewählt. Beim Index-Calculus-Algorithmus wird ein beliebiges Polynom (Körperelement) faktorisiert. Coppersmith verwendet dagegen zwei Polynome vom Grad $n^{2/3}$. Da diese einen kleineren Grad haben, steigt die Wahrscheinlichkeit, daß sie über S zerfallen.

Es sei $k \in \mathbb{N}$ so gewählt, daß $2^k \leq (n/\log_2(n))^{1/3}$ und setze $h := \lceil \frac{n}{2^k} \rceil$. Wähle $u_1(x)$ und $u_2(x)$ teilerfremd vom Grad kleiner s und berechne

$$w_1(x) = u_1(x) \cdot x^h + u_2(x) \quad (4.5)$$

und

$$w_2(x) \equiv w_1(x)^{2^k} \pmod{f(x)}. \quad (4.6)$$

Es gilt dann

$$\begin{aligned} w_2(x) &\equiv u_1(x)^{2^k} \cdot x^{h \cdot 2^k} + u_2(x)^{2^k} \pmod{f(x)} \\ &\equiv u_1(x)^{2^k} \cdot x^{h \cdot 2^k - n} \cdot f_1(x) + u_2(x)^{2^k} \pmod{f(x)}, \end{aligned}$$

da $x^n \equiv f_1(x) \pmod{f(x)}$ und das Potenzieren mit 2^k in Körpern der Charakteristik 2 ein Homomorphismus ist. Zerfallen beide Polynome $w_1(x)$ und $w_2(x)$ in irreduzible Polynome aus S , so erhalten wir, wenn wir den diskreten Logarithmus der Gleichung (4.6) nehmen, die Beziehung

$$\begin{aligned} \log_{\alpha(x)} w_2(x) &\equiv 2^k \cdot \log_{\alpha(x)} w_1(x) \pmod{2^n - 1} \\ \Leftrightarrow \log_{\alpha(x)} w_2(x) - 2^k \cdot \log_{\alpha(x)} w_1(x) &\equiv 0 \pmod{2^n - 1}. \end{aligned}$$

und damit eine lineare Gleichung für die Logarithmen der Elemente aus S .

Der Aufwand für die Aufstellung der Datenbasis ist $O(\exp((c + o(1))(n \log^2(n))^{1/3}))$ mit $c \approx 1.4$. Diese Laufzeitabschätzung stützt sich auf heuristische Annahmen.

2. Berechnung des Logarithmus

Es sei der Logarithmus des Elements $g(x)$ bezüglich der Basis $\alpha(x)$ gesucht. Falls $g(x)$ einen Grad $\deg(g(x)) \leq s$ hat, dann kann der Logarithmus aus der Datenbasis entnommen werden.

Es sei nun $\deg(g(x)) > s$. Wir gehen zunächst wie beim Index-Calculus-Algorithmus vor. Es wird $r \in \{1, \dots, n-1\}$ zufällig ausgewählt und

$$h(x) = g(x) \cdot \alpha^r(x)$$

berechnet. Wir faktorisieren $h(x)$ und überprüfen, ob es glatt bezüglich der Schranke $\tilde{s} := (n^2 \log_2(x))^{1/3}$ ist. Falls ja, dann ergibt sich der Logarithmus von $g(x)$ zu

$$\log_{\alpha(x)} g(x) = \sum_{i=1}^k \log_{\alpha(x)} p_i(x) - r.$$

Die Polynome $p_i(x)$ sind vom Grad kleiner oder gleich $\tilde{s}$. Die Logarithmen $\log_{\alpha(x)} p_i(x)$ der Polynome $p_i(x)$ mit $\deg p_i(x) \leq s$ stehen in der Datenbasis. Die noch fehlenden Logarithmen der Polynome $p_i(x)$ mit $s < \deg(p_i(x)) \leq \tilde{s}$ werden noch berechnet. Die Berechnung wird vereinfacht, indem man sie durch Polynome kleineren Grades ersetzt. Zur Bestimmung des Logarithmus von $p_i(x)$ wählen wir k , so daß $2^k \leq \sqrt{n/b}$ mit $b \approx (n^2 \log n)^{1/3} + \deg(p_i(x))/2$ und setzen $h := \lceil n/2^k \rceil$.

Anschließend wählt man zwei zufällige teilerfremde Polynome $u_1(x)$ und $u_2(x)$ vom Grad kleiner gleich $\tilde{s}$ und berechnen die Polynome

$$w_1(x) := u_1(x) \cdot x^h + u_2(x)$$

und

$$w_2(x) := w_1(x)^{2^k},$$

so daß $p_i(x) \mid w_1(x)$. Zerfallen $\frac{w_1(x)}{p_i(x)}$ und $w_2(x)$ in irreduzible Polynome kleineren Grades,

$$w_1(x) = p_i(x) \prod_{j=1}^k s_j(x)$$

und

$$w_2(x) = \prod_{j=1}^l t_j(x),$$

so erhält man

$$\begin{aligned} \log_{\alpha(x)} w_2(x) &\equiv 2^k \log_{\alpha(x)} w_1(x) \equiv \sum_{j=1}^l \log_{\alpha(x)} t_j(x) \pmod{2^n - 1} \\ &\equiv 2^k (\log_{\alpha(x)} p_i(x) + \sum_{j=1}^k \log_{\alpha(x)} s_j(x)) \pmod{2^n - 1} \\ \Rightarrow \log_{\alpha(x)} p_i(x) &\equiv 2^{-k} \sum_{j=1}^l \log_{\alpha(x)} t_j(x) - \sum_{j=1}^k \log_{\alpha(x)} s_j(x) \pmod{2^n - 1} \end{aligned}$$

Diese Gleichung reduziert die Berechnung der Logarithmen $\log_{\alpha(x)} p_i(x)$ auf die Berechnung der $\log_{\alpha(x)} s_j(x)$ und $\log_{\alpha(x)} t_j(x)$. Diese können entweder aus der Datenbasis entnommen werden, oder werden durch rekursive Anwendung des obigen Schrittes berechnet. Der Aufwand beträgt $O(\exp(c + o(1))(n \log^2 n)^{1/3})$ mit $c \approx 1,1$.

Kapitel 5

Authentifikation

Wenn sich Alice in einen Hostrechner einloggt, wie kann er Rechner wissen, daß es wirklich Alice ist? Wie kann er erkennen, daß Eve versucht sich als Alice auszugeben? Üblicherweise wird dieses Problem mit Hilfe eines Passworts gelöst. Alice gibt ihr Passwort ein und der Rechner überprüft, ob es korrekt ist. Alice und der Rechner haben ein gemeinsames Geheimnis, das der er jedesmal von Alice verlangt, wenn sie sich einloggen will.

5.1 Authentifikation mit Einwegfunktionen

Roger Needham und Mike Guy bemerkten, daß der Host gar nicht die Passwörter zu kennen braucht. Der Host muß nur die richtigen Passwörter von den falschen unterscheiden können. Dies kann einfach mit Einwegfunktionen realisiert werden. Der Host speichert nur die Werte $f(\text{Passwort})$, wobei f eine Einwegfunktion ist.

1. Alice sendet ihr Passwort dem Host.
2. Der Host wendet eine Einwegfunktion auf das Passwort an.
3. Der Host vergleicht das Ergebnis mit dem zuvor gespeicherten Wert.

Da jetzt der Host nicht die Passwörter speichert, sondern nur die Werte der mit einer Einwegfunktion verschlüsselten Passwörter, ist diese Liste für einen Angreifer nicht direkt verwendbar, da er daraus die Passwörter nicht mehr so leicht rekonstruieren kann.

Diese Authentifikation ist nicht sonderlich sicher, da ein *Wörterbuchangriff* unternommen werden kann. Eine Datei, die mit einer Einwegfunktion verschlüsselten Passwörter enthält, stellt immer noch einen Schwachpunkt dar. Mallory erstellt eine Liste der am häufigsten benutzten Passwörter, verschlüsselt jedes Passwort mit der Einwegfunktion und speichert die Ergebnisse. Mallory schafft es die Datei mit den verschlüsselten Passwörtern, die sich auf dem Hostrechner befindet, zu stehlen und sucht nach Übereinstimmungen in den beiden Dateien. Dieser Angriff ist erstaunlich erfolgreich, was nicht an dem Verfahren mit der Einwegfunktion liegt, sondern ist den unachtsamen Benutzern anzulasten, da sie einfach erratbare Passwörter wählen.

5.2 Authentifikation mit Hilfe von Public Key

Die Protokolle zur Authentifikation mit Hilfe von Passwörtern haben eine weitere enorme Sicherheitslücke. Wenn Alice das Passwort zu ihrem Host sendet, kann Eve ihr Passwort lesen, wenn sie Zugang zu dem Kommunikationsweg oder dem Prozessorspeicher des Hosts hat.

Mit Hilfe der Public-Key Kryptographie kann dieses Problem behoben werden. Der Host hat eine Datei mit den öffentlichen Schlüsseln aller Benutzer und alle Benutzer haben ihren öffentlichen Schlüssel. Ein einfaches Protokoll kann wie folgt aussehen:

1. Der Host sendet Alice eine Zufallszahl r .
2. Alice verschlüsselt r mit ihrem geheimen Schlüssel und schickt das Chifftrat an den Host.
3. Der Host entschlüsselt das Chifftrat mit dem öffentlichen Schlüssel von Alice und vergleicht es mit r .

Ein Angreifer kann sich jetzt nicht dem Host gegenüber als Alice identifizieren, da er keinen Zugang zu Alices geheimen Schlüssel hat. Alice sendet niemals ihren geheimen Schlüssel über den Kommunikationsweg zu dem Host. Eve, die die Interaktion zwischen Alice und dem Host abhört, kann keine Information gewinnen, die es ihr ermöglichen würde, Alices geheimen Schlüssel zu berechnen oder sich als Alice auszugeben, ohne daß es der Host bemerkt. Weder der Host noch der Kommunikationsweg müssen sicher sein.

Es ist für Alice nicht geschickt, eine beliebige Zufallszahl r , die sie zugeschickt bekommt, zu verschlüsseln. Eve könnte dadurch die an Alice geschickten Nachrichten entschlüsseln. Ein verbessertes Protokoll sieht wie folgt aus:

1. Alice berechnet etwas, wobei sie eine eigene Zufallszahl und ihren geheimen Schlüssel verwendet, und schickt das Ergebnis an den Host.
2. Der Host sendet Alice eine andere Zufallszahl.
3. Alice verwendet eine vorgegebene Funktion, um einen Funktionswert, der von den beiden Zufallszahlen und ihrem geheimen Schlüssel abhängt, zu berechnen und schickt das Ergebnis an den Host.
4. Der Host überprüft dies.

Falls Alice genauso wenig dem Host vertraut, wie der Host Alice traut, fordert sie ihn auf, sich ihr gegenüber zu authentifizieren.

5.3 Authentifikation mit Schlüsselaustausch

Diese Protokolle verbinden die Authentifikation mit einem Schlüsselaustausch, um ein generelles Problem zu lösen: Alice und Bob sind an zwei verschiedenen Enden eines Netzwerks und wollen sicher Daten übertragen. Wie können Alice und Bob einen geheimen Schlüssel austauschen und dabei sicher sein, daß keiner von ihnen in Wirklichkeit mit einem Angreifer Mallory spricht? Die meisten dieser Protokolle basieren auf der Tatsache, daß eine Schlüsselzentrale mit jedem Teilnehmer einen gemeinsamen geheimen Schlüssel besitzt und daß alle Schlüssel am richtigen Ort sind, bevor das Protokoll beginnt.

Wide-mouth frog

Das wide-mouth frog Protokoll ist vielleicht das einfachste Protokoll, das ein symmetrisches Verfahren und eine vertrauenswürdige Schlüsselzentrale benutzt. Alice und Bob besitzen jeweils einen geheimen Schlüssel A bzw. B mit der Schlüsselzentrale. Diese Schlüssel werden nur für den Schlüsselaustausch und nicht zur Verschlüsselung von eigentlichen Nachrichten verwendet. Mit Hilfe von nur zwei Nachrichten übermittelt Alice an Bob einen geheimen Sitzungsschlüssel K . Mit T_A und T_B werden Zeitstempel bezeichnet. Die Verschlüsselungsfunktionen E_A und E_B sind mit den geheimen Schlüsseln parametrisiert, die Alice und Bob mit der Schlüsselzentrale teilen.

1. Alice erzeugt das Tupel $(Alice, E_A(T_A, Bob, K))$ und schickt es an die Schlüsselzentrale.
2. Die Schlüsselzentrale entschlüsselt die Nachricht von Alice. Sie erzeugt einen neuen Zeitstempel T_B und sendet das verschlüsselte Tupel $E_B(T_B, Alice, K)$ an Bob.

Die wichtigste Annahme bei diesem Protokoll ist, daß Alice kompetent ist, gute (sichere) Sitzungsschlüssel zu erzeugen. Man muß beachten, daß es schwierig ist, Zufallszahlen zu erzeugen. Alice ist möglicherweise nicht dazu in der Lage.

Needham-Schroeder

Das Protokoll von Needham und Schroeder benutzt ein symmetrisches Verfahren und eine Schlüsselzentrale.

1. Alice erzeugt eine Zufallszahl R_A und schickt das Tupel $(Alice, Bob, R_A)$ an die Schlüsselzentrale.
2. Die Schlüsselzentrale erzeugt einen zufälligen Sitzungsschlüssel K und schickt das Tupel $E_A(R_A, Bob, K, E_B(K, Alice))$ an Alice.
3. Alice entschlüsselt und erhält den Sitzungsschlüssel K . Sie überprüft die Zufallszahl R_A und schickt das Tupel $E_B(K, Alice)$ an Bob.

4. Bob entschlüsselt und erhält den Sitzungsschlüssel K . Er erzeugt eine Zufallszahl R_B und schickt das Tupel $E_K(R_B)$ an Alice.
5. Alice entschlüsselt und erhält die Zufallszahl R_B . Sie berechnet $R_B - 1$ und schickt das Tupel $E_K(R_B - 1)$ an Bob.
6. Bob prüft, ob $R_B - 1$ geschickt wurde.

Kerberos

Das Kerberos-Protokoll ist eine Variante des Needham-Schroeder-Protokolls. Die Zufallszahl R_A wird durch einen Zeitstempel ersetzt.

1. Alice sendet das Tupel $(Alice, Bob)$ an die Schlüsselzentrale.
2. Die Schlüsselzentrale schickt das Tupel $(E_A(T_{SZ}, L, K, Bob), E_B(T_{SZ}, L, K, Alice))$ an Alice. Die Nachricht wird mit einer Gültigkeitsdauer L und einem Zeitstempel T_{SZ} versehen.
3. Alice schick das Tupel $(E_K(Alice, T_A), E_B(T_{SZ}, L, K, Alice))$ an Bob.
4. Bob sendet das Tupel $E_K(T_A + 1)$ an Alice.

Neuman-Stubblebine

1. Alice erzeugt eine Zufallszahl R_A und sendet das Tupel $(Alice, R_A)$ an Bob.
2. Bob erzeugt eine Zufallszahl R_B und sendet das Tupel $(Bob, R_B, E_B(Alice, R_A, T_B))$ mit einem Zeitstempel T_B versehen an die Schlüsselzentrale.
3. Die Schlüsselzentrale erzeugt einen zufälligen Sitzungsschlüssel K und sendet das Tupel $E_A(Bob, R_A, K, T_B), E_B(Bob, K, T_B), R_B$ an Alice.
4. Alice entschlüsselt und überprüft, ob R_A mit dem Wert im 1-ten Schritt übereinstimmt. Alice sendet $E_B(Alice, K, T_B), E_K(R_B)$ an Bob.
5. Bob entschlüsselt und erhält den Sitzungsschlüssel K . Er überprüft, ob T_B und R_B mit den Werten im 2-ten Schritt übereinstimmen.

5.4 Digital Cash

Ein ideales System für „Digital Cash“ soll folgende sechs Eigenschaften erfüllen:

- *Unabhängigkeit*: das digitale Bargeld kann über Netze verschickt und gespeichert werden.

- *Sicherheit*: das digitale Bargeld kann nicht kopiert und mehrfach benutzt werden.
- *Nichzurückverfolgbarkeit*: Die Privatsphäre des Benutzers ist gewahrt. Keiner kann feststellen, welche Transaktionen vom Benutzer vorgenommen wurden.
- *Off-line Bezahlung*: Bei der Bezahlung mit dem digitalen Bargeld ist keine Kommunikation mit der Bank notwendig. Die Bezahlung erfolgt off-line.
- *Übertragbarkeit*: Das digitale Bargeld kann an andere Benutzer übertragen werden.
- *Teilbarkeit*: Das digitale Bargeld kann in kleinere Beträge aufgeteilt werden.

Das erste vorgeschlagene Protokoll, das alle diese Punkte erfüllte, benötigt 200 MByte Datentransfer für eine Buchung. Mitterweile existieren Protokolle, die nur 20 KByte Datentransfer brauchen und in wenigen Sekunden arbeiten.

Im folgenden präsentieren wir ein Protokoll, das die ersten vier Eigenschaft erfüllt. Wir werden das Protokoll schrittweise ausbauen, um Sicherheitslücken zu beseitigen.

5.4.1 Erste Protokolle

1. Protokoll

1. Alice stellt 100 anonyme Bankschecks über jeweils 1000 DM aus.
2. Alice steckt jeden Bankscheck zusammen mit Durchschlagspapier in einen Umschlag und trägt dann alle Umschläge zur Bank.
3. Die Bank öffnet 99 Umschläge und prüft, ob der richtige Betrag (1000 DM) auf jedem Scheck von Alice eingetragen wurde.
4. Die Bank unterzeichnet den letzten ungeöffneten Umschlag. Die Unterschrift der Bank wird durch das Durchschlagspapier auf den Scheck übertragen. Die Bank gibt den ungeöffneten Umschlag an Alice zurück und belastet ihr Konto mit 1000 DM.
5. Alice öffnet den Umschlag und verwendet den Scheck bei einem Händler.
6. Der Händler überprüft die Unterschrift der Bank und akzeptiert den Scheck.
7. Der Händler bringt den Scheck zur Bank.
8. Die Bank überprüft ihre Unterschrift und zahlt 1000 DM dem Händler aus.

Bei diesem Protokoll kann die Bank nicht feststellen, von wem der Scheck kommt. Die Wahrscheinlichkeit, daß ein Betrug von der Bank unentdeckt bleibt, beträgt durch die 1-aus-100-Wahl lediglich 1/100. Durch die Einführung einer Geldstrafe für Betrug lohnt es sich nicht für Alice zu betrügen.

Allerdings gibt es noch eine Sicherheitslücke, da Alice oder der Händler den Scheck kopieren und mehrfach verwenden könnten. Mit dem folgenden Protokoll soll diese Sicherheitslücke beseitigt werden.

2. Protokoll

1. Alice stellt 100 anonyme Bankschecks über jeweils 1000 DM aus. Auf jeden Scheck trägt sie eine Zufallszahl ein, die lang genug ist, um die Wahrscheinlichkeit, daß eine andere Person dieselbe Zufallszahl benutzt, verschwindend klein ist.
2. Alice steckt jeden Bankscheck zusammen mit Durchschlagspapier in einen Umschlag und trägt alle Umschläge zur Bank.
3. Die Bank öffnet 99 Umschläge und prüft, ob der richtige Betrag (1000 DM) und eine jeweils andere Zufallszahl auf jedem Scheck von Alice eingetragen wurde.
4. Die Bank unterzeichnet den letzten ungeöffneten Umschlag. Die Unterschrift der Bank wird durch das Durchschlagspapier auf den Scheck übertragen. Die Bank gibt den ungeöffneten Umschlag an Alice zurück und belastet ihr Konto mit 1000 DM.
5. Alice öffnet den Umschlag und verwendet den Scheck bei einem Händler.
6. Der Händler überprüft die Unterschrift der Bank und akzeptiert den Scheck.
7. Der Händler bringt den Scheck zur Bank.
8. Die Bank überprüft ihre Unterschrift und schaut in einer Datenbank nach, ob ein Scheck mit derselben Zufallszahl eingelöst wurde. Falls nicht, dann zahlt die Bank dem Händler 1000 DM aus. Die Bank speichert die Zufallszahl in ihrer Datenbank ab.
9. Falls aber ein Scheck mit derselben Zufallszahl eingelöst wurde, dann akzeptiert die Bank den Scheck nicht.

Die Bank kann jetzt zwar einen Betrug feststellen, kann aber den Betrüger nicht identifizieren. Der Händler ist im Nachteil, da er nicht feststellen kann, ob Alice ihm eine Kopie des Schecks aushändigt.

3. Protokoll

1. Alice stellt 100 anonyme Bankschecks über jeweils 1000 DM aus. Auf jeden Scheck trägt sie eine Zufallszahl ein, die lang genug ist, um die Wahrscheinlichkeit, daß eine andere Person dieselbe Zufallszahl benutzt, verschwindend klein ist.
2. Alice steckt jeden Bankscheck zusammen mit Durchschlagspapier in einen Umschlag und trägt alle Umschläge zur Bank.

3. Die Bank öffnet 99 Umschläge und prüft, ob der richtige Betrag (1000 DM) und eine jeweils andere Zufallszahl auf jedem Scheck von Alice eingetragen wurde.
4. Die Bank unterzeichnet den letzten ungeöffneten Umschlag. Die Unterschrift der Bank wird durch das Durchschlagspapier auf den Scheck übertragen. Die Bank gibt den ungeöffneten Umschlag an Alice zurück und belastet ihr Konto mit 1000 DM.
5. Alice öffnet den Umschlag und verwendet den Scheck bei einem Händler.
6. Der Händler überprüft die Unterschrift der Bank.
7. Der Händler fordert Alice auf einen Zufallsstring auf den Scheck zu schreiben.
8. Alice tut dies.
9. Der Händler bringt den Scheck zur Bank.
10. Die Bank überprüft ihre Unterschrift und schaut in einer Datenbank nach, ob ein Scheck mit derselben Zufallszahl eingelöst wurde. Falls nicht, dann zahlt die Bank dem Händler 1000 DM aus. Die Bank speichert die Zufallszahl und den Zufallsstring in ihrer Datenbank ab.
11. Falls aber ein Scheck mit derselben Zufallszahl eingelöst wurde, dann akzeptiert die Bank den Scheck nicht. Sie überprüft den Zufallsstring auf dem Scheck mit dem in der Datenbank. Falls sie übereinstimmen, weiß die Bank, daß der Händler den Scheck kopiert hat. Falls sie verschieden sind, dann weiß die Bank, daß die Person, die den Scheck gekauft hat, ihn kopiert hat.

Damit ist ein Betrug des Händlers, der sich der Bank zu erkennen gibt, nicht mehr möglich, die anonyme Alice ist aber in der Lage zu betrügen. Um Alices Anonymität zu sichern, solange sie sich korrekt verhält, ihre Identität aber offenzulegen, sobald sie versucht, zu betrügen bedarf es einiger Hilfsmittel, die jetzt bereitgestellt werden. Weiterhin ist noch eine elektronische Version der „Unterschrift mit Durchschlagspapier“ zu finden.

5.4.2 Hilfsprotokolle

Blinde Unterschrift

Die Bank soll eine Nachricht m von Alice ungesehen unterschreiben. Für die Signatur wird das RSA-Signaturverfahren benutzt. Die Bank verwendet ihren geheimen Schlüssel d . Der öffentliche Schlüssel (e, n) ist allgemein zugänglich.

Alice berechnet

$$t := m \cdot k^e \pmod{n},$$

wobei k eine Zufallszahl ist. Die Bank signiert t , indem sie t^d berechnet und das Ergebnis Alice liefert. Alice berechnet

$$s := t^d \cdot k^{-1} \equiv m^d \cdot (k^e)^d \cdot k^{-1} \equiv m^d \cdot k^1 \cdot k^{-1} \equiv m^d \pmod{n}$$

und erhält die mit dem geheimen Schlüssel der Bank signierter Nachricht m^d . Die Bank kennt die Nachricht m nicht.

Die blinde Unterschrift entspricht dem Unterschreiben des Umschlags durch die Bank, wobei die Unterschrift durch das Durchschlagspapier auf den Scheck übertragen wird.

Verschleiern einer Nachricht

Alice wählt eine Zufallszahl k und veröffentlicht $(f(I, k), k)$, wobei f eine bekannte Einwegfunktion ist. Jeder kann nach der Bekanntgabe von I verifizieren, daß auch wirklich $f(I, k)$ veröffentlicht wurde.

Mit dieser Technik läßt sich das Problem, die Anonymität einer ehrlichen Alice zu sichern und bei einem Betrug doch die Identität des Betrügers zu erhalten:

Alice spaltet eine Bitfolge I , die ihre Identität vollständig beschreibt (Name, Adresse und alle anderen Informationen, die die Bank verlangt), in zwei Bitfolgen derart auf, daß $I = I_L \oplus I_R$ gilt. Dann veröffentlicht Alice einige dieser „halbierten Identitäten“ in der Form $(f(I_{i_L}, k_{i_1}), k_{i_1})$ und $(f(I_{i_R}, k_{i_2}), k_{i_2})$. Kann man es erreichen, daß bei korrektem Verhalten von jedem Paar nur eine Hälfte offengelegt wird, bei einem Betrug jedoch von mindestens einem Paar beide Teile, so ist diese scheinbar widersprüchliche Anforderung von Anonymität und Bekanntgabe der Identität erfüllt.

5.4.3 Ein vollständiges Protokoll

1. Alice stellt n anonyme Bankschecks über einen festen, jeweils identischen Betrag aus. Auf jeden Scheck trägt sie eine genügend lange Zufallszahl ein. Auf jedem Scheck wird m mal der Identitätsstring von Alice eingetragen, jeweils auf verschiedene Weisen halbiert und verschleiert (s.o.).
2. Alle n Schecks werden von Alice für eine blinde Unterschrift der Bank vorbereitet.
3. Alice öffnet nach Wahl der Bank $n - 1$ dieser Schecks; die Bank prüft den Betrag, die Zufallszahl und alle (von Alice auch offengelegten) Identitätsstrings.
4. Die Bank signiert den letzten Scheck, gibt ihn Alice und belastet ihr Konto.
5. Alice entfernt die Verschleierung und verwendet den Scheck bei einem Händler.
6. Der Händler überprüft die Unterschrift der Bank.
7. Der Händler fordert Alice auf von den m Identitätsstrings jeweils, nach seiner Wahl, die linke oder rechte Hälfte bekanntzugeben.

8. Alice tut dies.
9. Der Händler bringt den Scheck zur Bank.
10. Die Bank überprüft ihre Unterschrift und schaut in einer Datenbank nach, ob ein Scheck mit derselben Zufallszahl eingelöst wurde. Falls nicht, dann zahlt die Bank dem Händler den Betrag aus. Die Bank speichert die Zufallszahl in ihrer Datenbank ab.
11. Falls aber ein Scheck mit derselben Zufallszahl eingelöst wurde, dann akzeptiert die Bank den Scheck nicht. Sie vergleicht die offengelegten Hälften der Identitätsstrings mit denen des Schecks in der Datenbank. Falls sie alle übereinstimmen, weiß die Bank, daß der Händler den Scheck kopiert hat. Falls sie verschieden sind, dann weiß die Bank, daß die Person, die den Scheck gekauft hat, ihn kopiert hat. Da der zweite Händler höchst wahrscheinlich eine andere Kombination der halbierten Identitäten hat offenlegen lassen, kennt die Bank jetzt von einem der Identitätsstrings beide Hälften und damit die Identität des Betrügers.

Literaturverzeichnis

- [1] D. Coppersmith, A.M. Odlyzko, R. Schroepel; *Discrete Logarithms in $GF(p)$* ; Algorithmica, 1986; S. 1–15.
- [2] D. Denning; *Cryptography and Data Security*; Addison Wesley, London, 1983.
- [3] A.M. Odlyzko; *Discrete logarithms in finite fields and their cryptographic significance*; S. 224–314.
- [4] F. Schaefer-Lorinser; *Algorithmische Zahlentheorie*; Vorlesungsskript, Universität Karlsruhe.
- [5] B. Schneier; *Applied Cryptography*; 2. Ausgabe, Wiley, 1996.
- [6] J. Seberry, J. Pieprzyk; *Cryptography*; Prentice Hall, New York, 1989.
- [7] G. J. Simmons (Ed.); *Contemporary Cryptology*; IEEE Press, 1992.
- [8] D. R. STinson; *Cryptography, Theory and Practice*; CRC Press, Boca Taton, Florida, 1995.